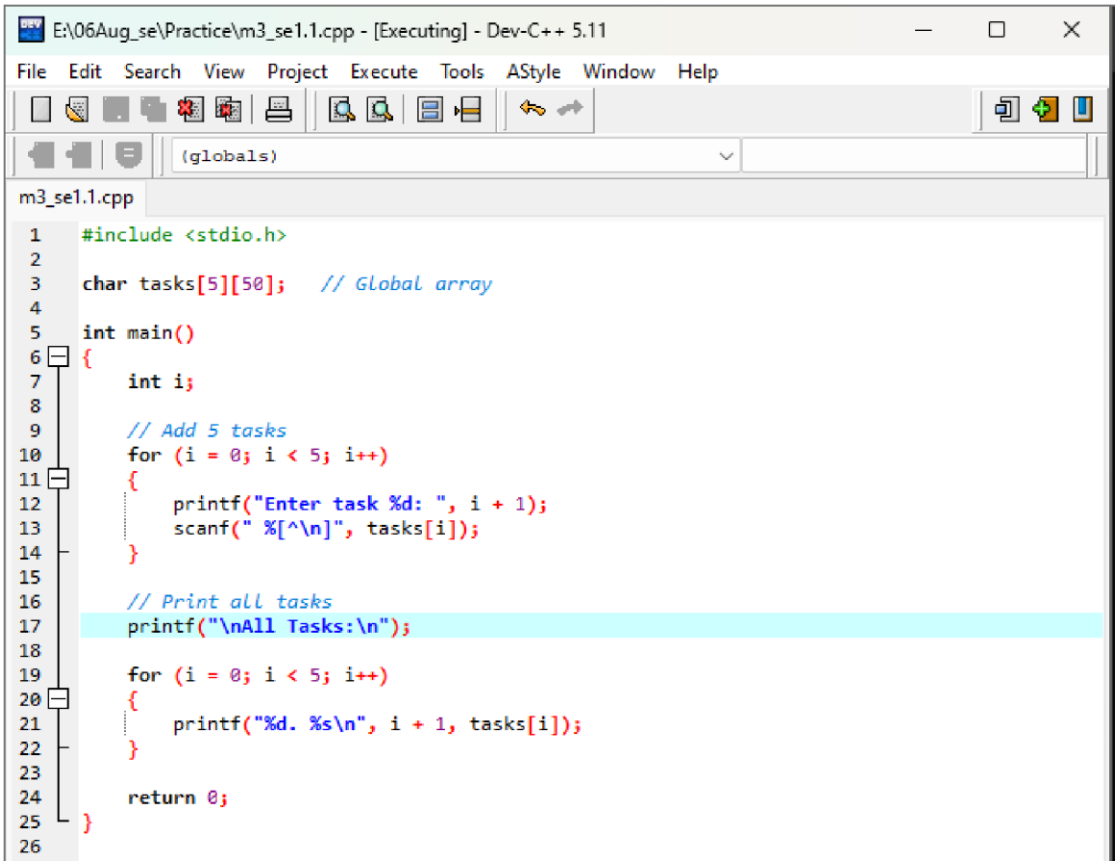
Modeual-3

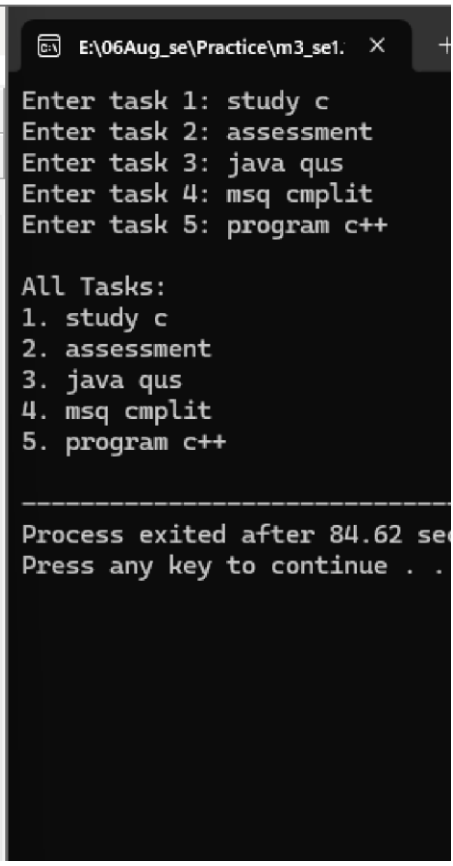
Introduction to oops programming

Session-1

Q.1 Write a simple C program tasklist_c.c that allows you to add up to 5 tasks (as strings) to a global array and print all tasks using a for loop.

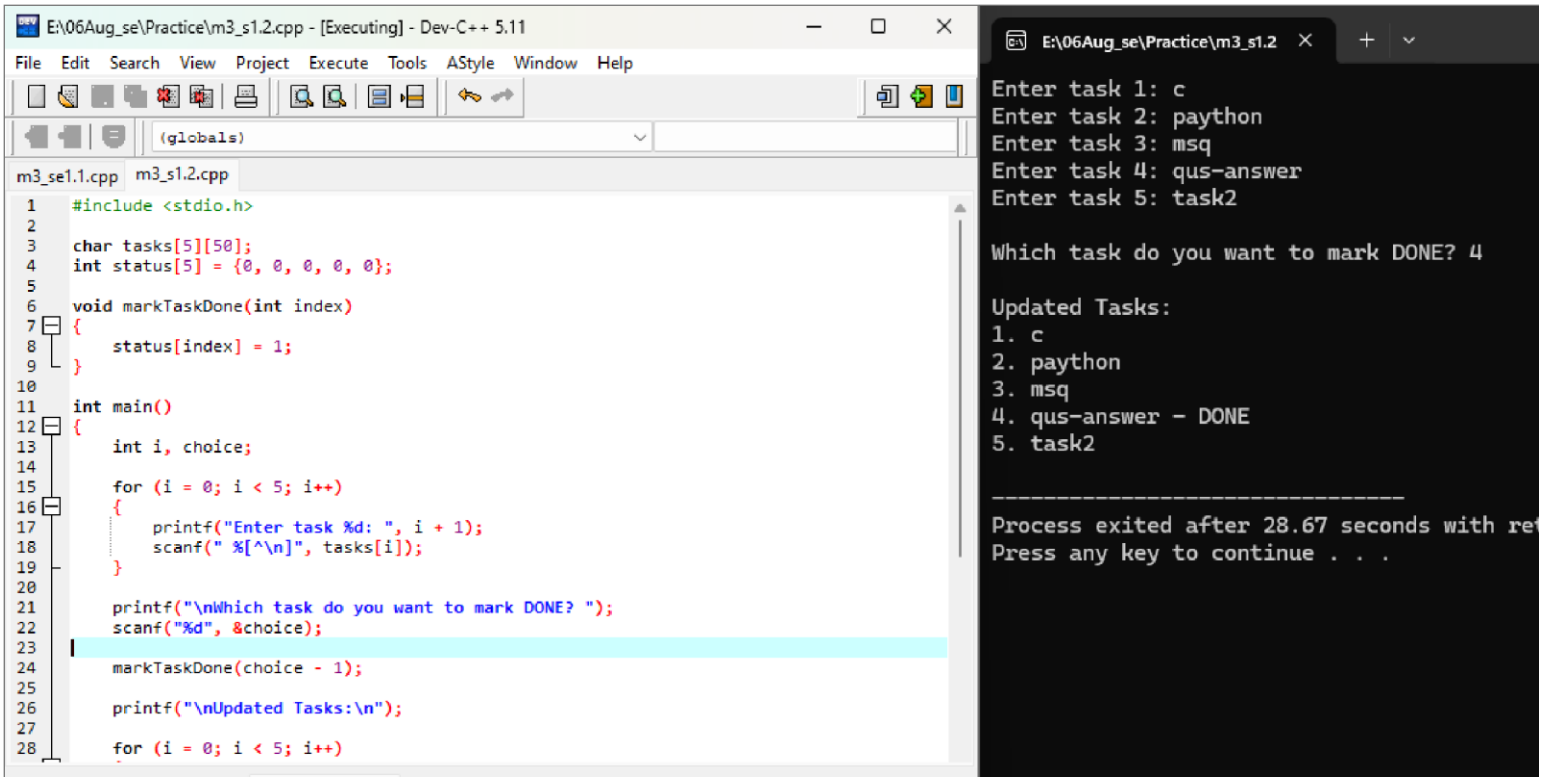
Ans.





Q.2 Modify your tasklist_c.c to add a function markTaskDone(int index) that sets the selected task to 'DONE' in the array, then print the updated list.

Hint: Use a separate status array or append ' - DONE' to the task string. Ans.




```
#include <stdio.h>

char tasks[5][50]; int
status[5] = {0, 0, 0, 0, 0};

void markTaskDone(int index)
{
    status[index] = 1;
}

int main()
```

```
{     int i,
choice;

    for (i = 0; i < 5; i++)
    {
        printf("Enter task %d: ", i + 1);
scanf(" %[^\n]", tasks[i]);
    }

    printf("\nWhich task do you want to mark DONE? ");
scanf("%d", &choice);

    markTaskDone(choice - 1);

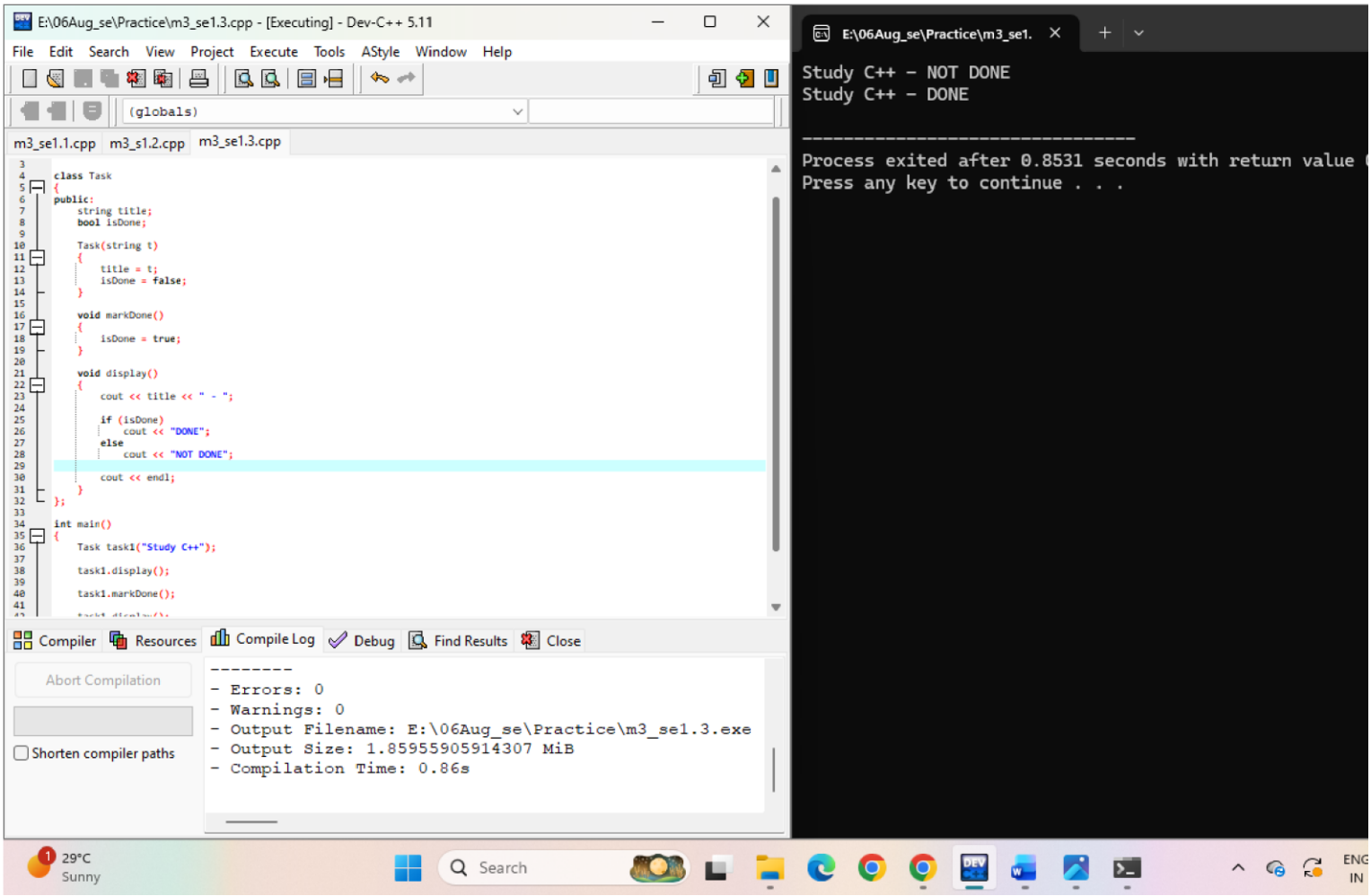
    printf("\nUpdated Tasks:\n");

    for (i = 0; i < 5; i++)
    {
        printf("%d. %s", i + 1, tasks[i]);
         if (status[i] == 1)
printf(" - DONE");

printf("\n");
    }

    return 0;
}
```

Q 3. Create a Task class in your favorite OOP language (Java, Python, or C++) with properties: title and isDone. Add a method markDone() that sets isDone to true, and a method display() that prints the task with its status. Ans.




```
#include <iostream>
using namespace std;

class Task
{
public:
    string title;
bool isDone;
Task(string t)
    {       title = t;
isDone = false;
```

```
    }

    void markDone()
    {
        isDone = true;
    }

    void display()
    {         cout << title <<
" - ";

        if (isDone)
cout << "DONE";
else
            cout << "NOT DONE";

        cout << endl;
    }
};

int main()
{
    Task task1("Study C++");

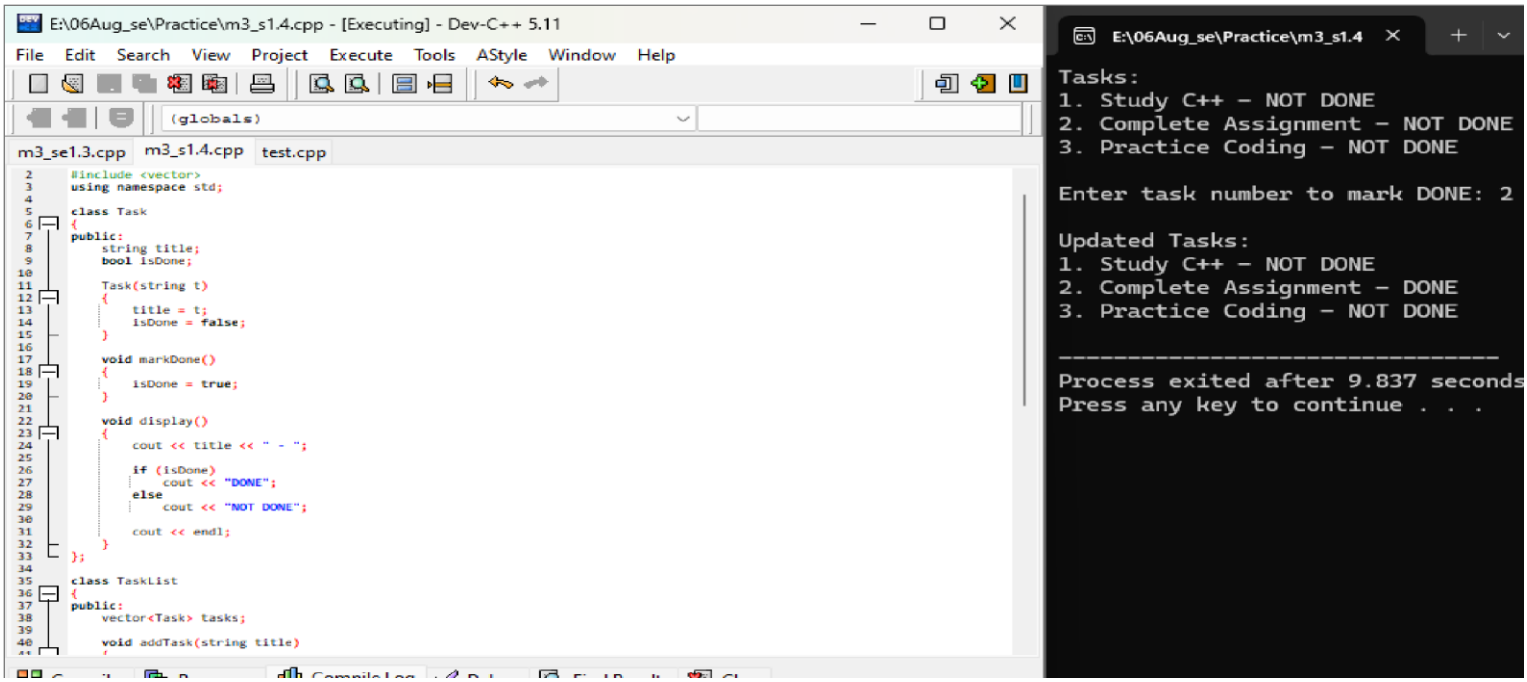
    task1.display();

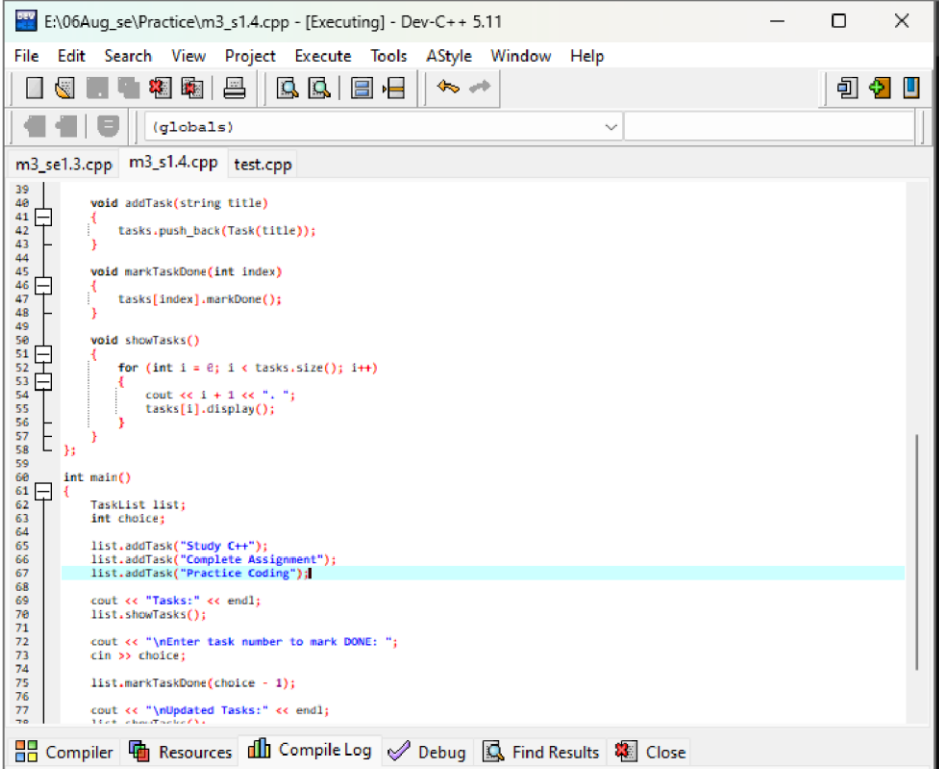
    task1.markDone();     task1.display();

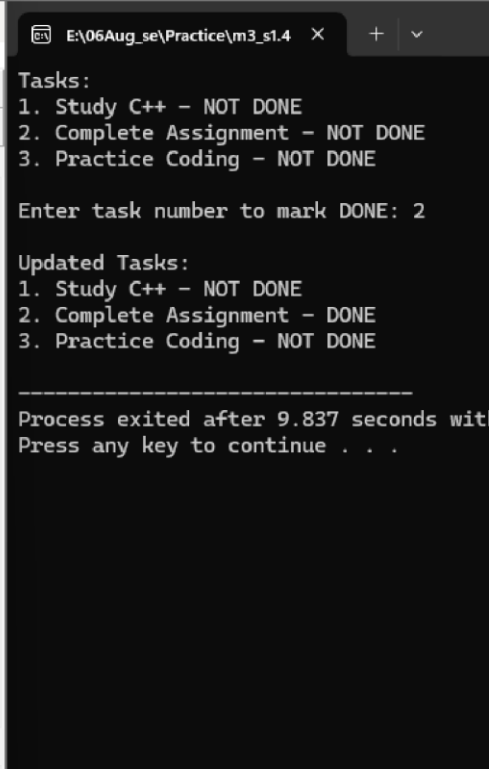
    return 0;
}
```

Q 4. Build a simple TaskList class/object that stores multiple Task objects and provides addTask(title), markTaskDone(index), and showTasks() methods. Demonstrate adding 3 tasks, marking one as done, and displaying all tasks with their statuses.

Ans.








```
#include <iostream>
#include <vector> using
namespace std;

class Task
{
public:
string title;
bool isDone;

    Task(string t)
    {       title = t;
isDone = false;
    }

    void markDone()
    {
        isDone = true;
```

```
    }

    void display()
    {        cout << title <<
" - ";

        if (isDone)
cout << "DONE";
else
            cout << "NOT DONE";

        cout << endl;
    }
};

class TaskList
{
public:
    vector<Task> tasks;

    void addTask(string title)
    {
        tasks.push_back(Task(title));
    }

    void markTaskDone(int index)
    {
        tasks[index].markDone();
```

```
    }

    void showTasks()
    {        for (int i = 0; i < tasks.size();
i++)
        {            cout << i + 1
<< ". ";
tasks[i].display();
        }
    }
};

int main()
{
    TaskList list;
int choice;

    list.addTask("Study                C++");
list.addTask("Complete     Assignment");
list.addTask("Practice Coding");

    cout << "Tasks:" << endl;
list.showTasks();

    cout << "\nEnter task number to mark DONE: ";
cin >> choice;

    list.markTaskDone(choice - 1);
```

```
    cout << "\nUpdated Tasks:" << endl;
list.showTasks();

    return 0;
```

Q.5 Compare your procedural C version and OOP TaskList version: List 3 problems you faced in the C version that were solved by using classes and methods in the OOP version.

Ans.

Data Management: In the C version, it was difficult to manage task data using separate variables and functions.

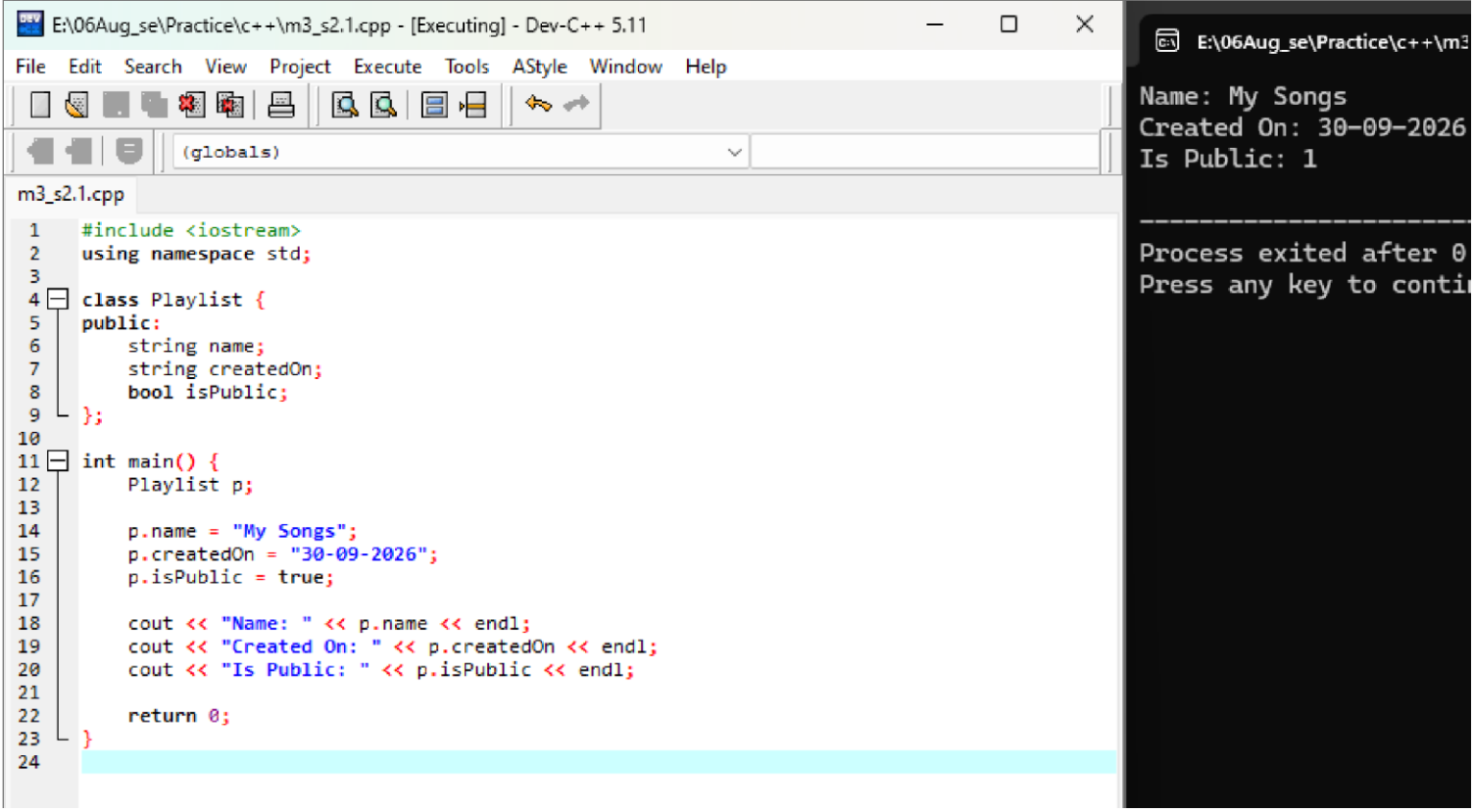
In the OOP version, the TaskList class keeps all task data together.

Code Organization: In the C version, the code was spread across different functions, making it harder to manage.

In the OOP version, classes and methods make the code more organized and easy to understand.

Data Protection: In the C version, data could be accessed directly.

In the OOP version, classes provide **e**ncapsulation, which keeps the data protected and allows it to be accessed through methods.

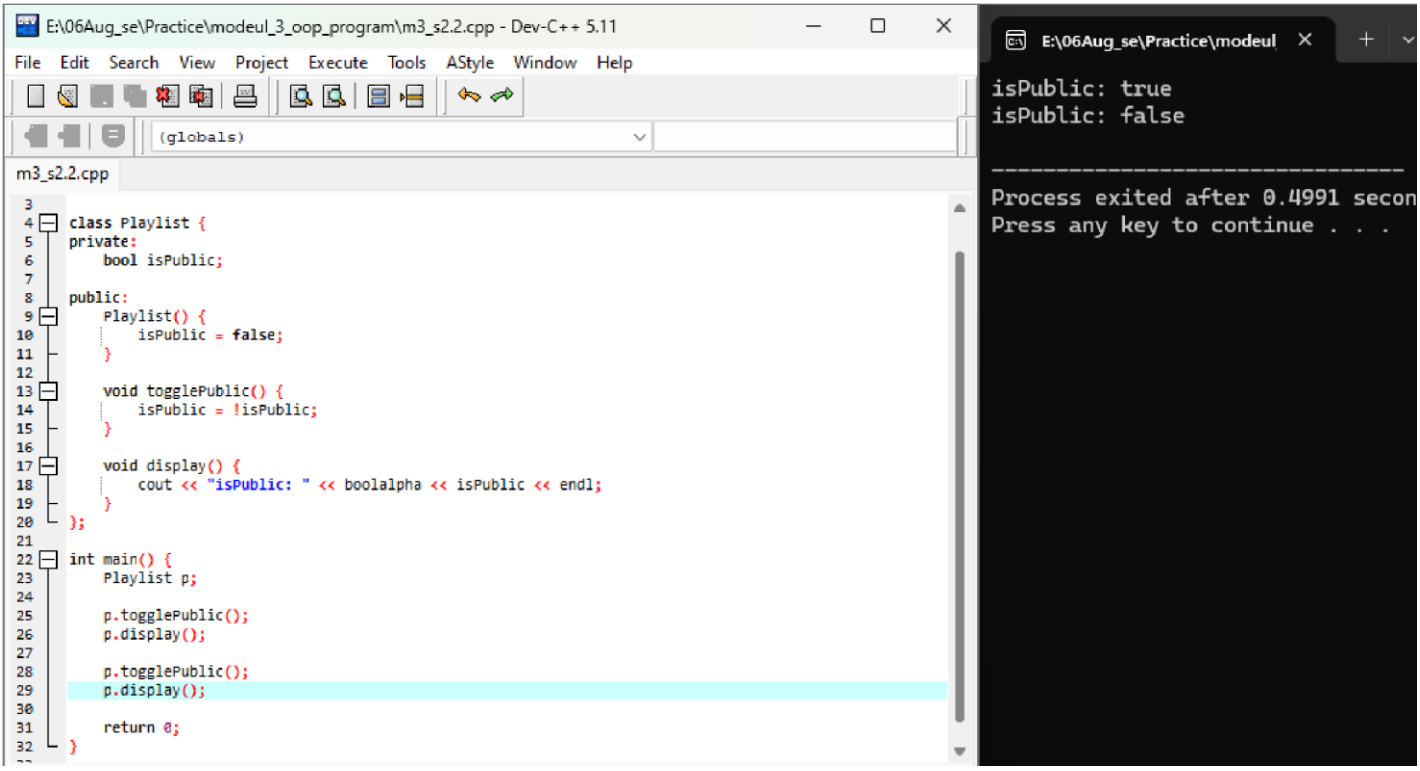


Session-2

Q 1. Create a class called Playlist with properties: name (string), createdOn (date), and is Public (boolean). Instantiate an object of Playlist and print all its properties.

Ans.

Q 2. Add a member function to your Playlist class named togglePublic () that switches is Public between true and false



each time it is called. Demonstrate by toggling the value twice and printing the result each time.

Ans.

```
#include <iostream> using
namespace std;

class Playlist {
private:     bool
isPublic;

public:
    Playlist() {
isPublic = false;
```

```
    }

    void togglePublic() {         isPublic = !isPublic;
    }

    void display() {
        cout << "isPublic: " << boolalpha << isPublic << endl;
    }
};

int main() {
    Playlist p;

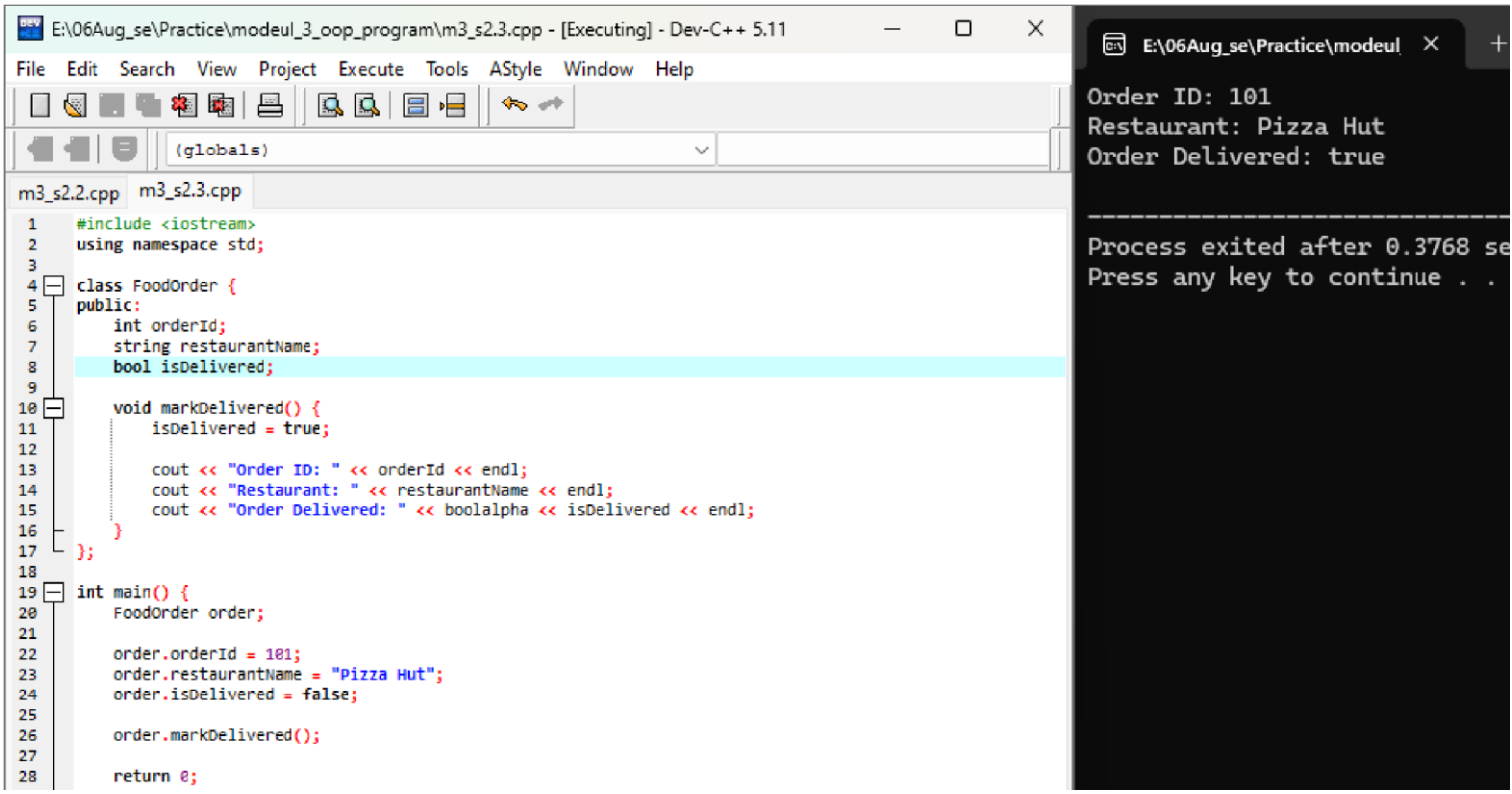
    p.togglePublic();
    p.display();

    p.togglePublic();
    p.display();

    return 0;
}
```

Q 3. Create a class called FoodOrder with properties: orderId (number), restaurantName (string), and isDelivered (boolean). Write a member function markDelivered() that sets isDelivered to true and prints a message. Instantiate FoodOrder and call markDelivered().

Ans.



#include <iostream> using namespace std;

class FoodOrder { public:
int orderId; string restaurantName; bool isDelivered;

```
    void markDelivered() {
isDelivered = true;

        cout << "Order ID: " << orderId << endl;         cout
<< "Restaurant: " << restaurantName << endl;         cout
<< "Order Delivered: " << boolalpha << isDelivered <<
endl;
    }
};

int main() {
    FoodOrder order;

    order.orderId = 101;
    order.restaurantName = "Pizza Hut";
order.isDelivered = false;

    order.markDelivered();

    return 0;
}
```

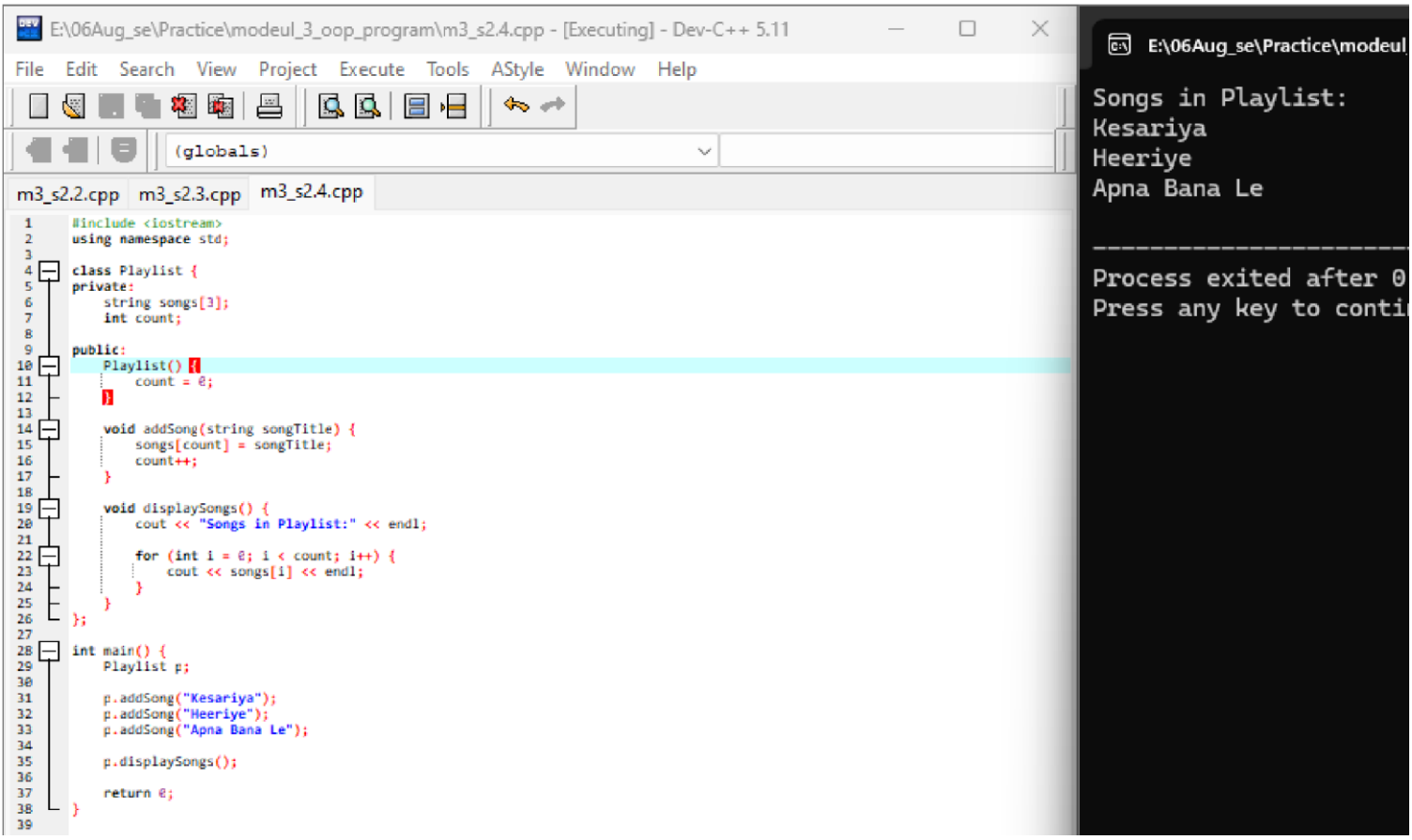
Q 4. Extend your Playlist class to include a member function addSong(songTitle) that adds the song title to an array property called songs. Demonstrate by adding

three song titles and displaying the updated songs list.

Hint: Initialize songs as an empty array inside the constructor.

Ans

```
#include <iostream> using
namespace std;
```




```
class Playlist {
private:
    string songs[3];
int count;
```

```
public:
    Playlist() {
count = 0;
    }

    void addSong(string songTitle) {
songs[count] = songTitle;
count++;
    }

    void displaySongs() {
        cout << "Songs in Playlist:" << endl;

        for (int i = 0; i < count; i++) {
cout << songs[i] << endl;
        }
    }
};

int main() {
    Playlist p;

    p.addSong("Kesariya");
```

```
        p.addSong("Heeriye");
        p.addSong("Apna Bana Le");

        p.displaySongs();

        return 0;
    }
```

Q 5. Refactor your FoodOrder class so that the constructor takes an object as a parameter (with keys orderId, restaurantName, isDelivered) instead of separate arguments. Update your instantiation code to use this new constructor signature.

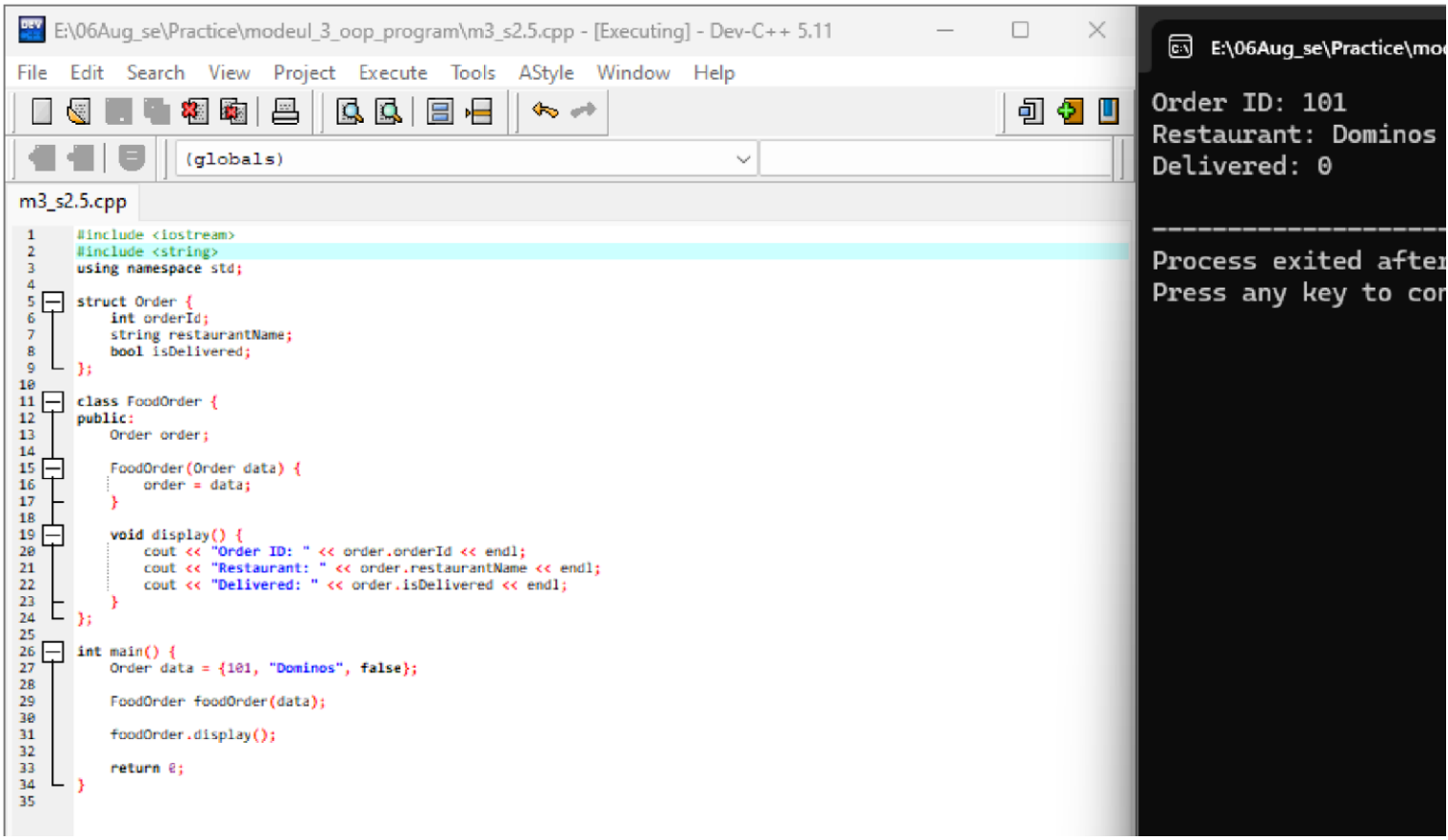
Ans.

#include <iostream>

#include <string> using

namespace std;

struct Order {



int orderId;

string

restaurantName

; bool

isDelivered;

};

```
class FoodOrder { public:
    Order order;

    FoodOrder(Order data) {
order = data;
    }

    void display() {
        cout << "Order ID: " << order.orderId << endl;
cout << "Restaurant: " << order.restaurantName
<< endl;
        cout << "Delivered: " << order.isDelivered << endl;
    }
};

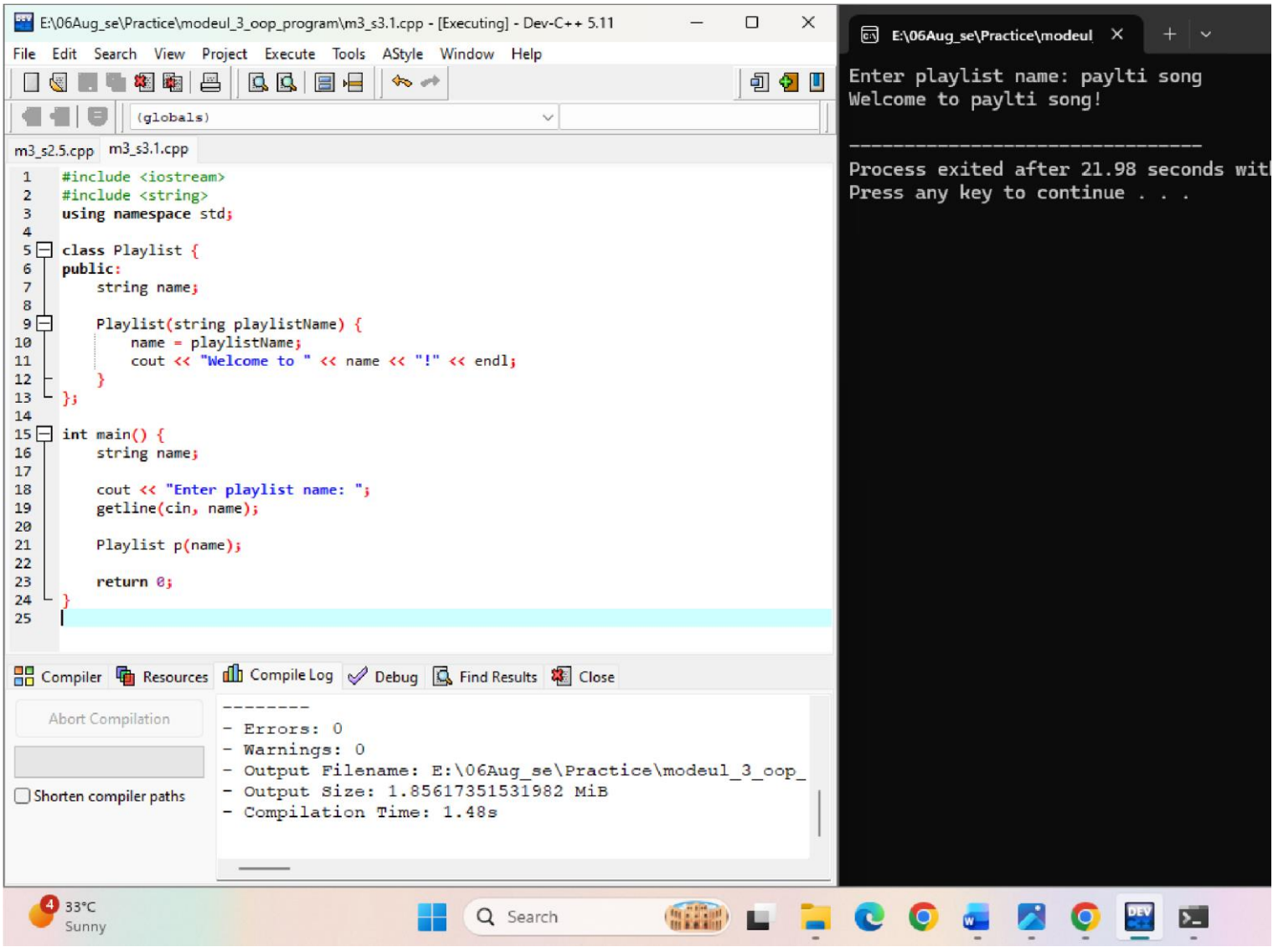
int main() {
    Order data = {101, "Dominos", false};
    FoodOrder foodOrder(data);

    foodOrder.display();

    return 0;
}
```

Session-3

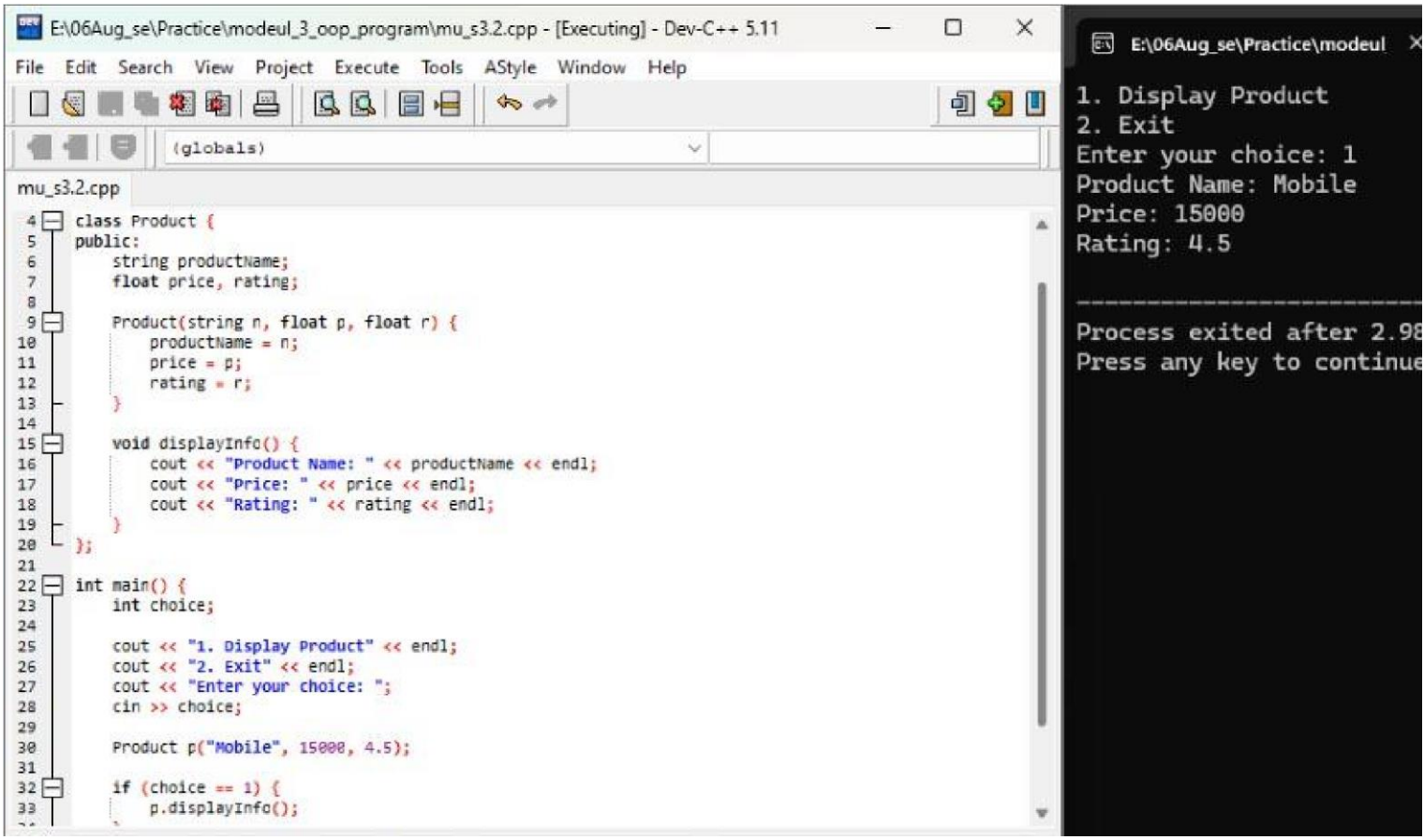
Q 1. Create a class called Playlist with a default constructor that sets the playlist name to 'My Favourites' and prints a welcome message when an object is created.



Ans.

Q 2. Build a class Product for a Flipkart-style app with a parameterized constructor that takes productName, price, and rating as arguments and displays all details using a displayInfo() method.

Ans.




```
#include <iostream> using

namespace std;

class Product {

public:

    string productName;

    float price, rating;
```

```
    Product(string n, float p, float r) {
productName = n;          price = p;
rating = r;
    }

    void displayInfo() {
        cout << "Product Name: " << productName <<
endl;
        cout << "Price: " << price << endl;
cout << "Rating: " << rating << endl;
    }
};

int main() {
int choice;

    cout << "1. Display Product" << endl;
cout << "2. Exit" << endl;     cout <<
"Enter your choice: ";
    cin >> choice;

    Product p("Mobile", 15000, 4.5);
```

```
    if (choice == 1) {
        p.displayInfo();
    }

    return 0;
}
```

Q 3. Create a class Movie with a parameterized constructor and then use a copy constructor to duplicate a Movie object. Print both the original and copied movie details to show they are identical.

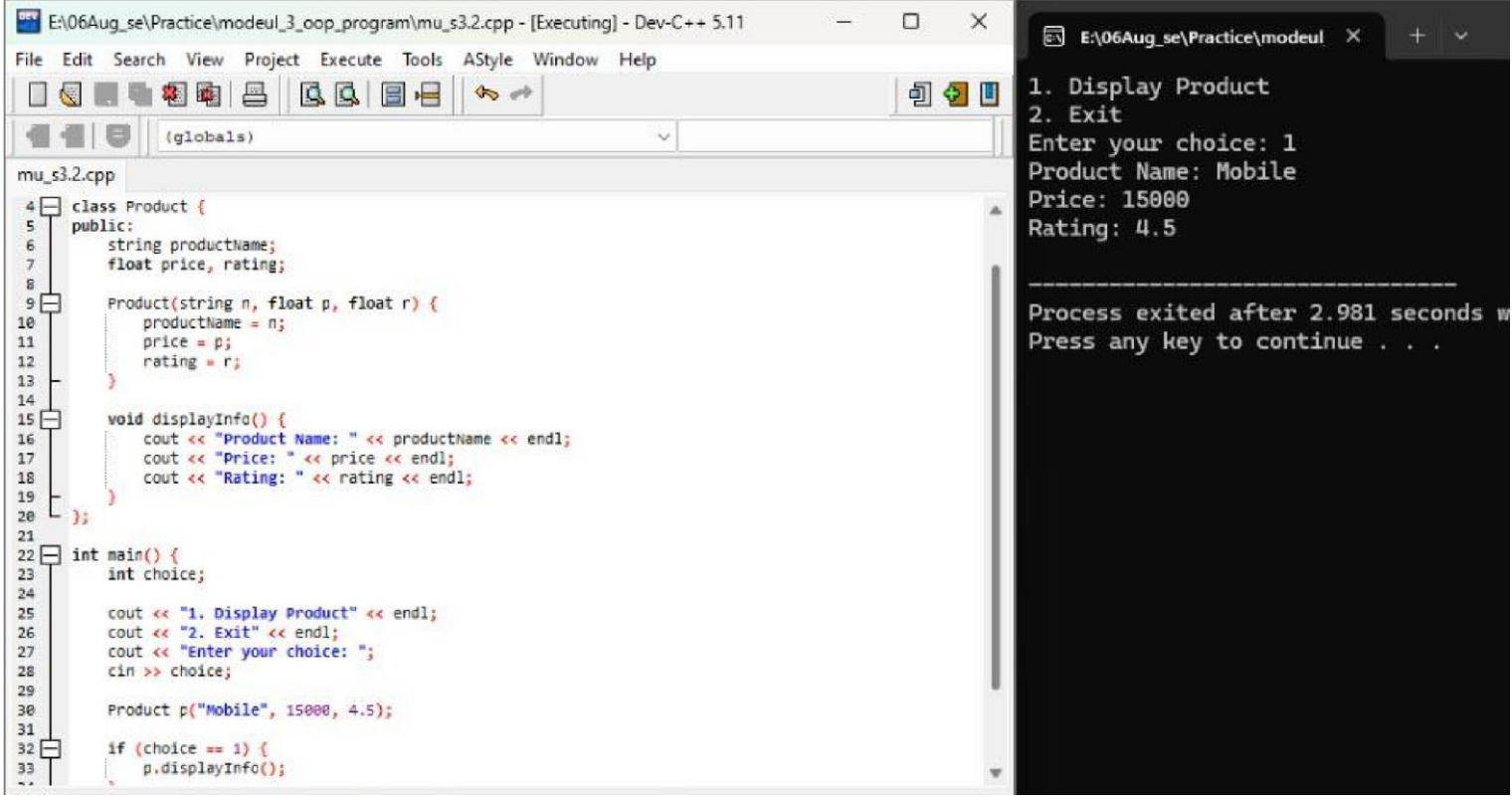
Ans.

```
#include <iostream> using

namespace std;

class Product {

public:
```



string productName;

float price, rating;

Product(string n, float p, float r) {

productName = n; price = p;

rating = r;

}

void displayInfo() {

cout << "Product Name: " << productName << endl;

cout << "Price: " << price << endl;

cout << "Rating: " << rating << endl;

}

};

```
int main() {
int choice;

    cout << "1. Display Product" << endl;
cout << "2. Exit" << endl;     cout <<
"Enter your choice: ";     cin >> choice;

    Product p("Mobile", 15000, 4.5);

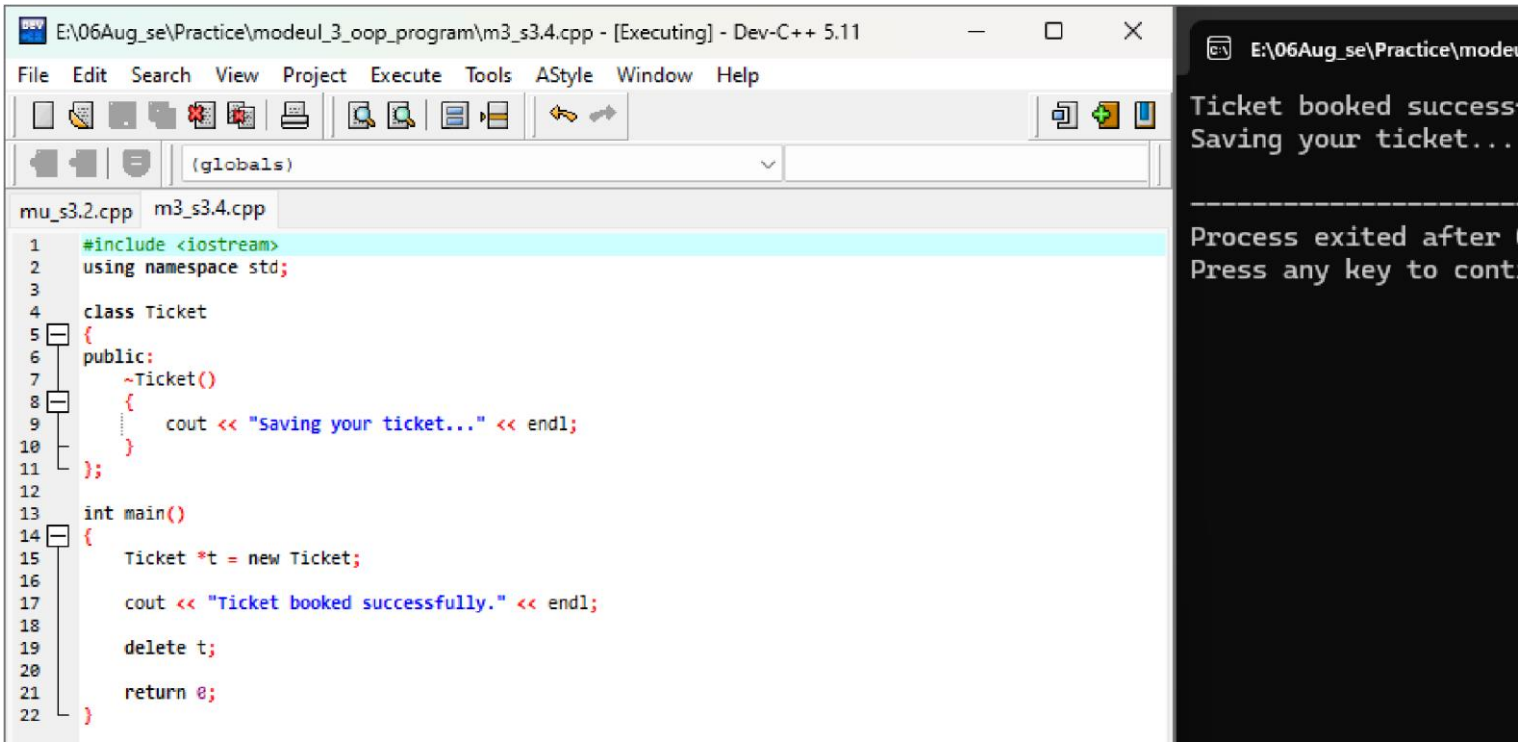
    if (choice == 1) {
        p.displayInfo();
    }
    return 0;
}
```

Q 4. Simulate a BookMyShow ticket booking by creating a Ticket class that prints 'Saving your ticket...' in its destructor. Create and delete a Ticket object to demonstrate the destructor lifecycle.

Hint: Use the __del__ method (Python) or ~Ticket() (C++) to define the destructor. Ans.

Q 5. Modify your Playlist class so that it auto-saves the playlist name to a file called autosave.txt when the object is destroyed, simulating an auto-save feature like Spotify.

Hint: Write



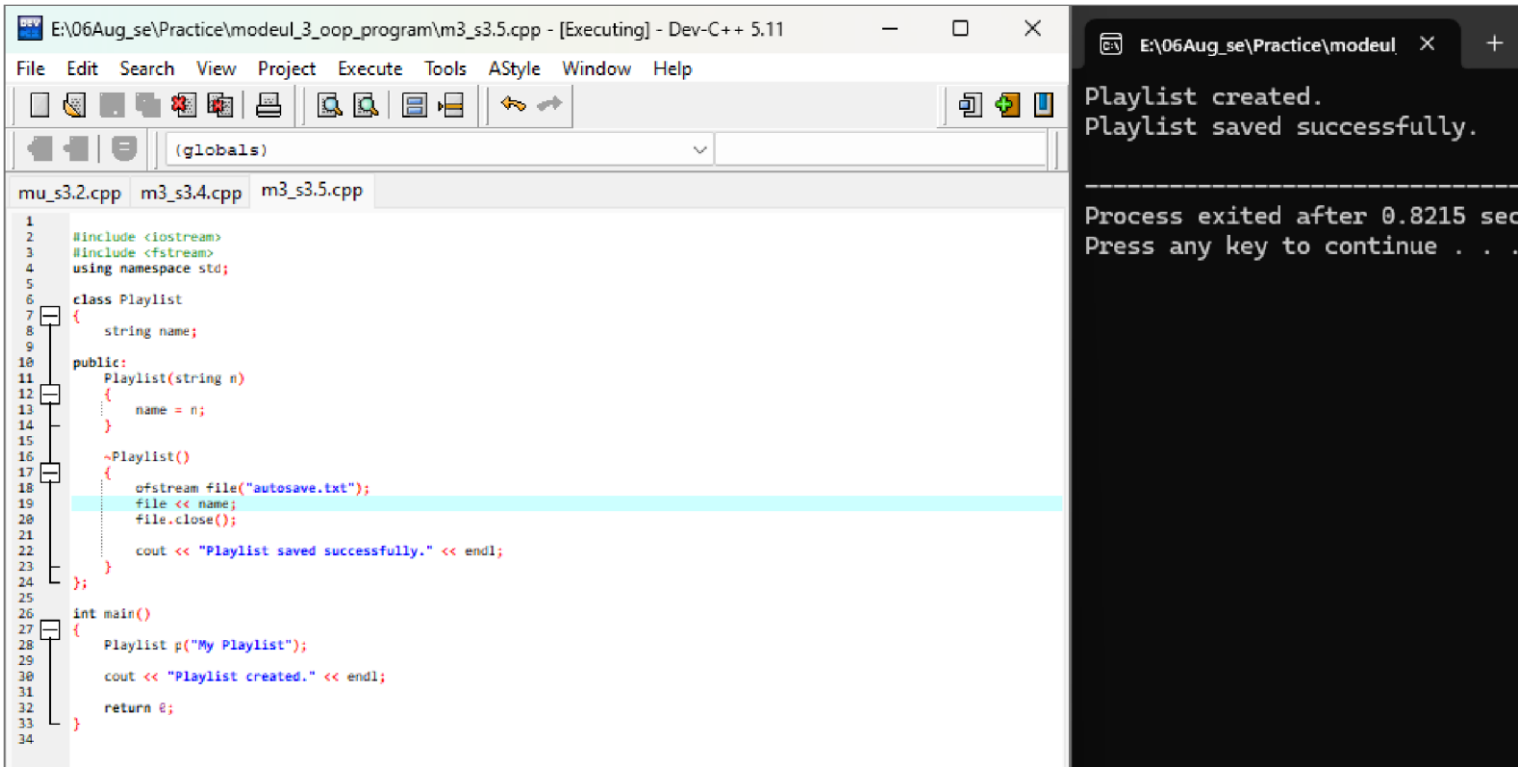
the file-saving code inside the destructor. Ans.

#include <iostream>

#include <fstream> using

namespace std;

class Playlist



{

string name;

public:

Playlist(string n)

{

name = n;

```
    }

    ~Playlist()
    {
        ofstream file("autosave.txt");
file << name;         file.close();

        cout << "Playlist saved successfully." << endl;
    }
};

int main()
{
    Playlist p("My Playlist");

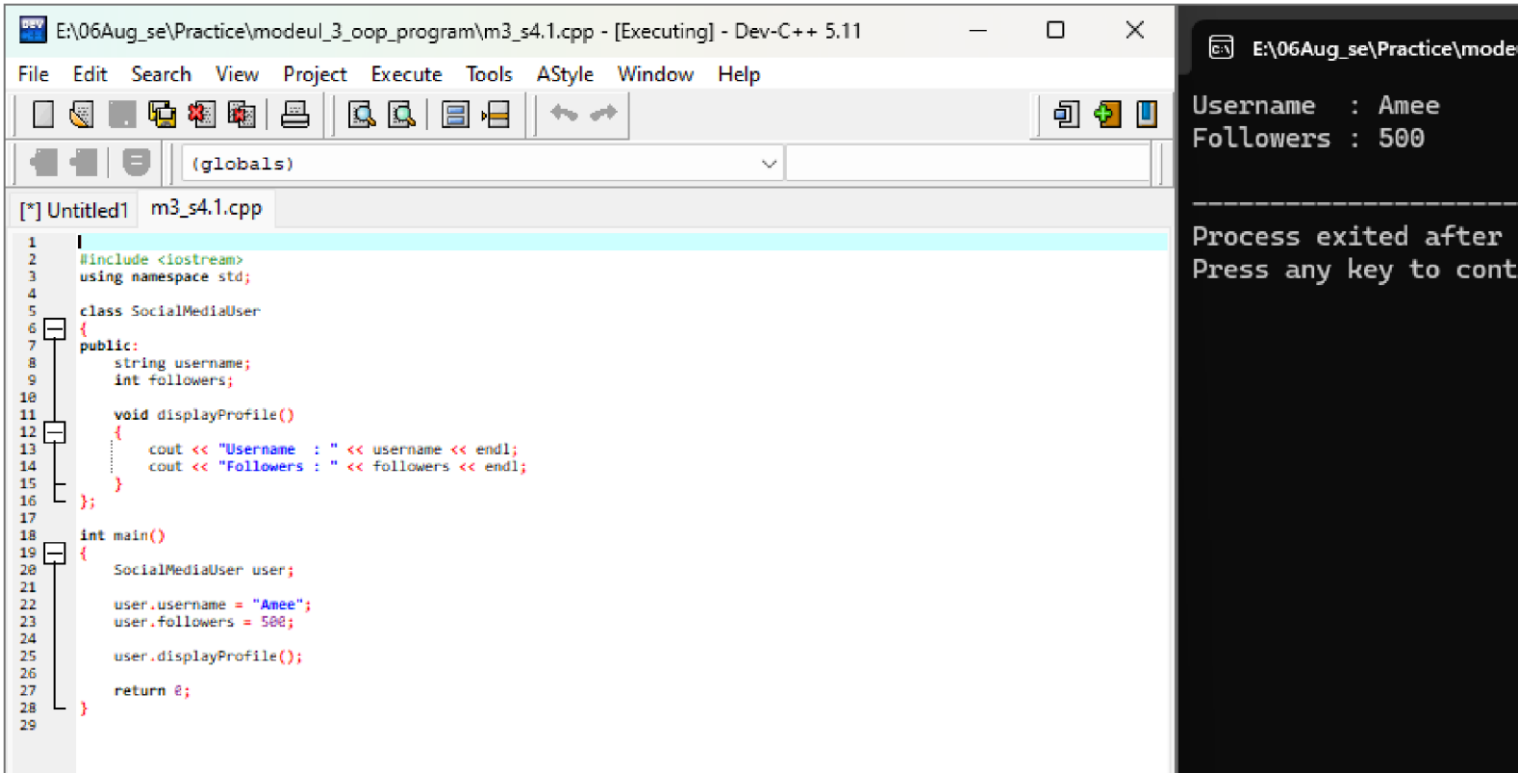
    cout << "Playlist created." << endl;

    return 0;
}
```

Session-4

Q 1. Create a base class called SocialMediaUser with properties username and followers, and a method displayProfile() that prints the username and follower count.

Ans.



Q 2. Build a YouTuber class that inherits from SocialMediaUser and adds a property channelName and a method uploadVideo(title) that prints 'Video [title] uploaded to [channelName]'.

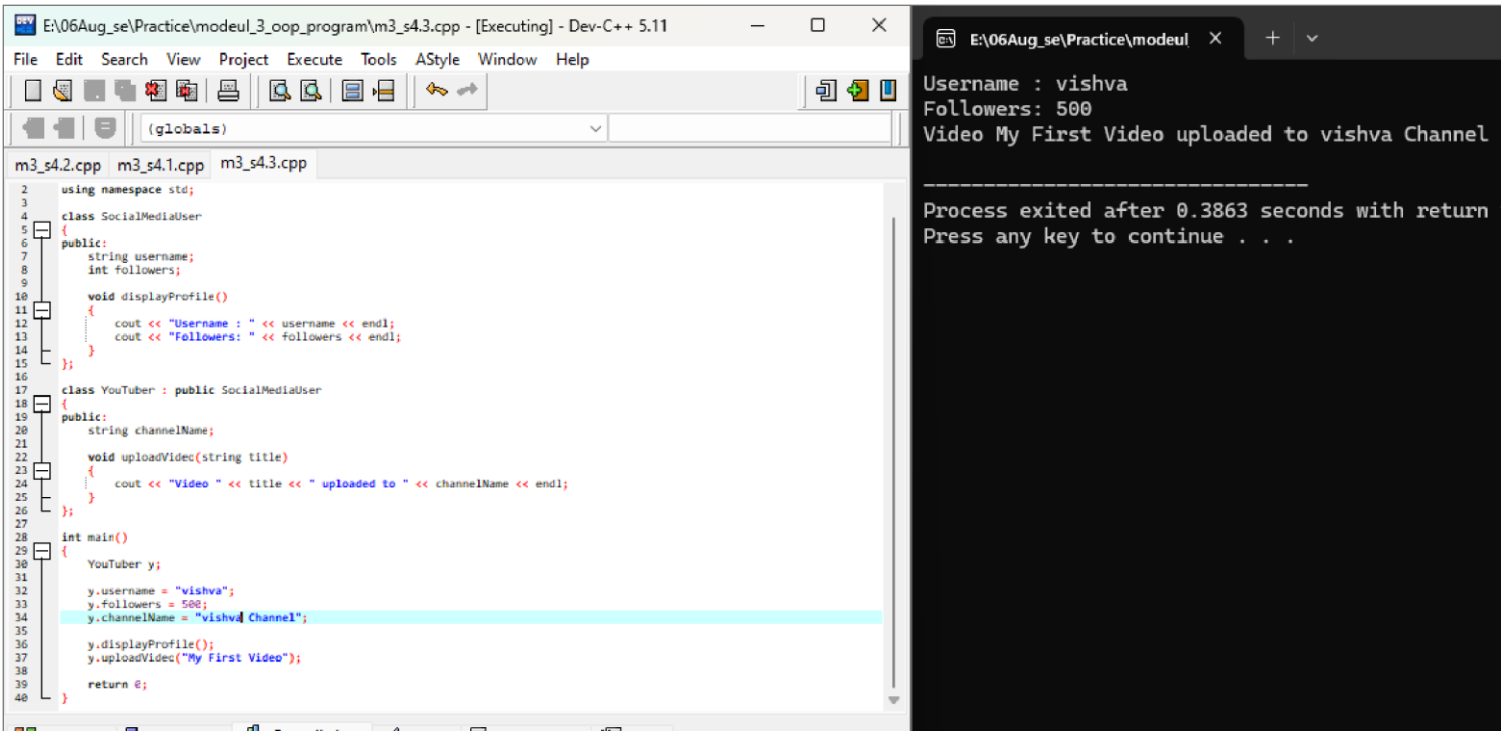
Ans.

#include <iostream> using

namespace std;

class SocialMediaUser

{



public:

string username;

int followers;

void displayProfile()

{

cout << "Username : " << username << endl;

cout << "Followers: " << followers << endl;

```
    }
};

class YouTuber : public SocialMediaUser
{
public:
    string channelName;

    void uploadVideo(string title)
    {
        cout << "Video " << title << " uploaded to " <<
channelName << endl;
    }
};

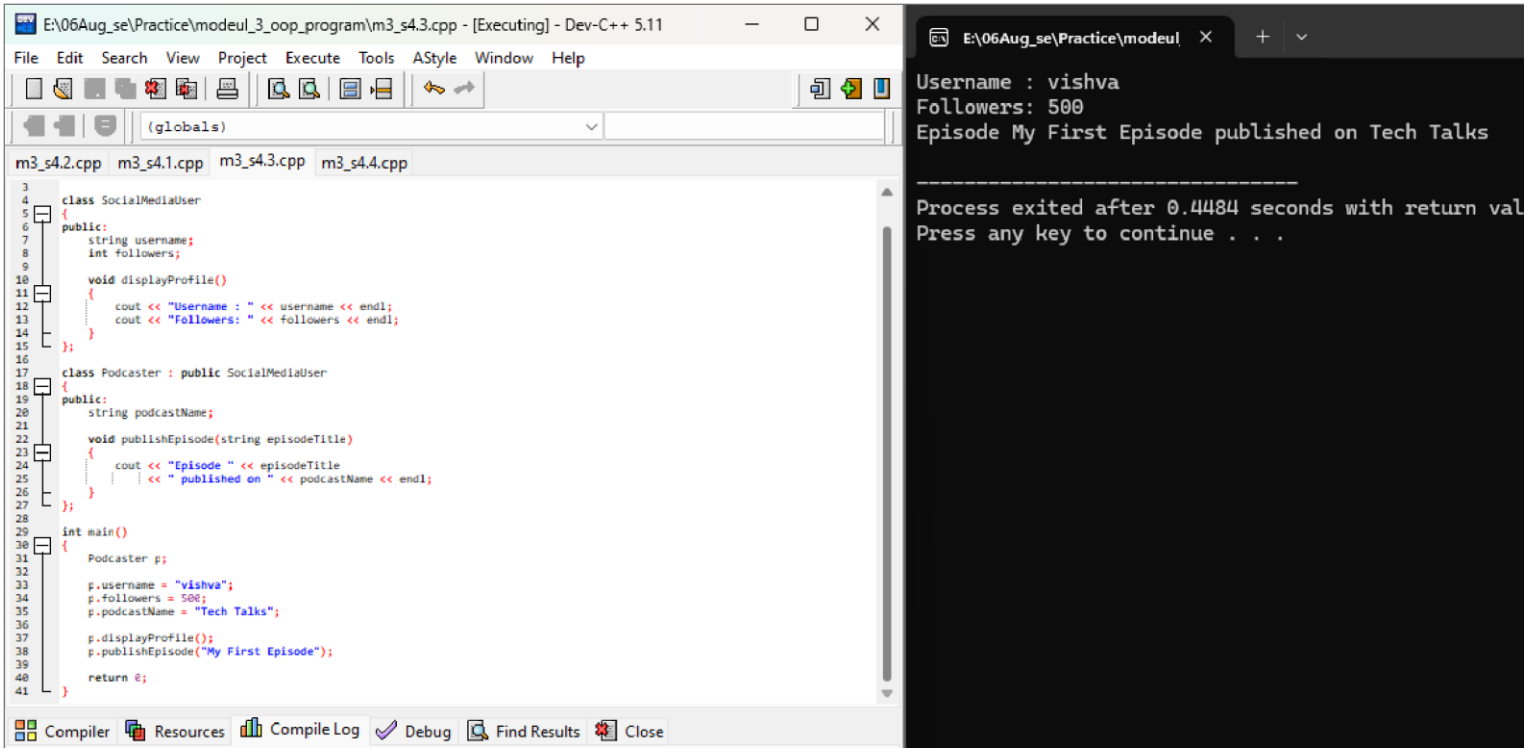
int main()
{
    YouTuber y;

    y.username = "vishva";
    y.followers = 500;
    y.channelName = "vishva Channel";

    y.displayProfile();
```

```
    y.uploadVideo("My First Video");

    return 0;
}
```



Q 3. Create a Podcaster class that also inherits from SocialMediaUser and adds a property podcastName and a method publishEpisode(episodeTitle) that prints 'Episode [episodeTitle] published on [podcastName]'.

Ans.

```
#include <iostream> using
namespace std;

class SocialMediaUser
{
public:
    string username;
    int followers;

    void displayProfile()
    {
        cout << "Username : " << username << endl;
cout << "Followers: " << followers << endl;
    }
};

class Podcaster : public SocialMediaUser
{
public:
    string podcastName;

    void publishEpisode(string episodeTitle)
    {
```

```
        cout << "Episode " << episodeTitle
             << " published on " << podcastName << endl;
    }
};

int main()
{
    Podcaster p;

    p.username = "vishva";
    p.followers = 500;
    p.podcastName = "Tech Talks";

    p.displayProfile();
    p.publishEpisode("My First Episode");

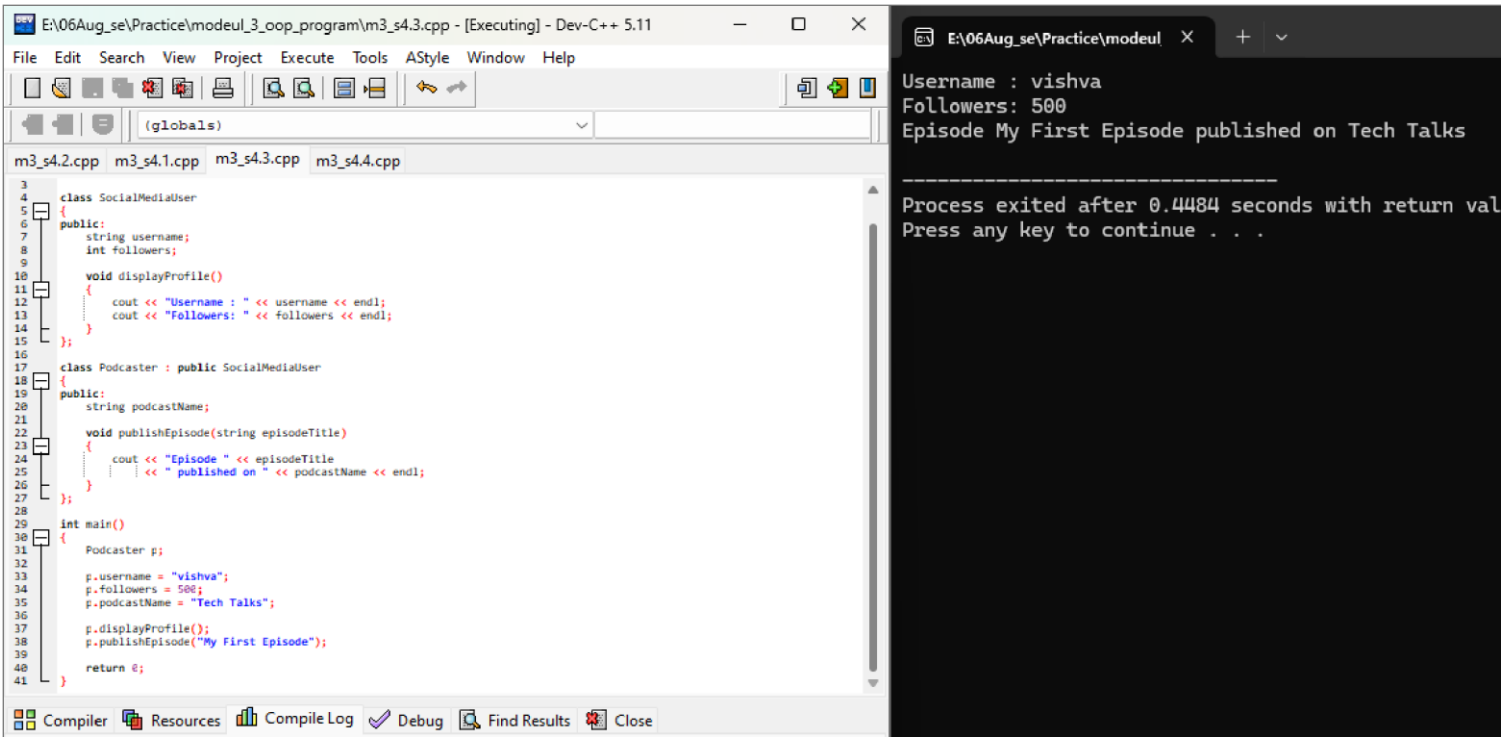
    return 0;
}
```

Q 4. Demonstrate multilevel inheritance by creating a class GamingYouTuber that inherits from YouTuber and adds a method streamGame(gameName) which prints

'[username] is now streaming [gameName] on [channelName]'.

Ans.

```
#include <iostream>
using namespace std;
```




```
class SocialMediaUser
{
public:
    string username;
    int followers;

    void displayProfile()
    {
```

```
        cout << "Username : " << username << endl;
cout << "Followers: " << followers << endl;
    }
};

class Podcaster : public SocialMediaUser
{
public:
    string podcastName;

    void publishEpisode(string episodeTitle)
    {
        cout << "Episode " << episodeTitle
             << " published on " << podcastName << endl;
    }
};

int main()
{
    Podcaster p;

    p.username = "Amee";
    p.followers = 500;
```

```
    p.podcastName = "Tech Talks";

    p.displayProfile();
    p.publishEpisode("My First Episode");

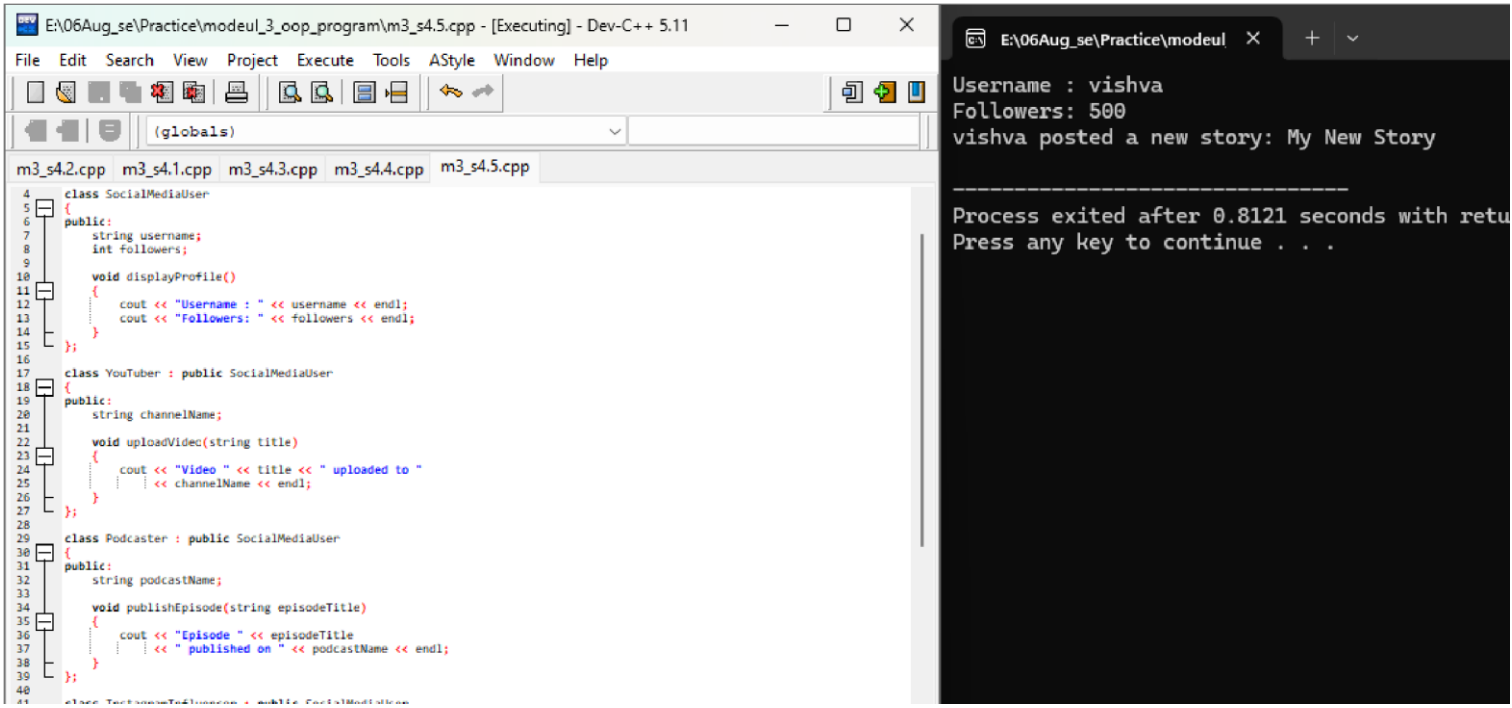
    return 0;
}
```

Q 5. Implement hierarchical inheritance by creating an InstagramInfluencer class that inherits from SocialMediaUser and adds a method postStory(storyTitle) which prints '[username] posted a new story: [storyTitle]'.

Hint: Think about how SocialMediaUser is the parent for YouTuber, Podcaster, and InstagramInfluencer. Ans.

```
#include <iostream>
using namespace std;

class SocialMediaUser
{
public:
    string username;
int followers;
```




```
    void displayProfile()
    {
        cout << "Username : " << username << endl;
cout << "Followers: " << followers << endl;
    }
```

```
};

class YouTuber : public SocialMediaUser
{
public:
    string channelName;

    void uploadVideo(string title)
    {
        cout << "Video " << title << " uploaded to "
             << channelName << endl;
    }
};

class Podcaster : public SocialMediaUser
{
public:
    string podcastName;

    void publishEpisode(string episodeTitle)
    {
        cout << "Episode " << episodeTitle
             << " published on " << podcastName << endl;
```

```
    }
};

class InstagramInfluencer : public SocialMediaUser
{
public:     void postStory(string
storyTitle)
    {
        cout << username << " posted a new story: "
             << storyTitle << endl;
    }
};

int main()
{
    InstagramInfluencer i;

    i.username = "vishva";
    i.followers = 500;

    i.displayProfile();
    i.postStory("My New Story");
```

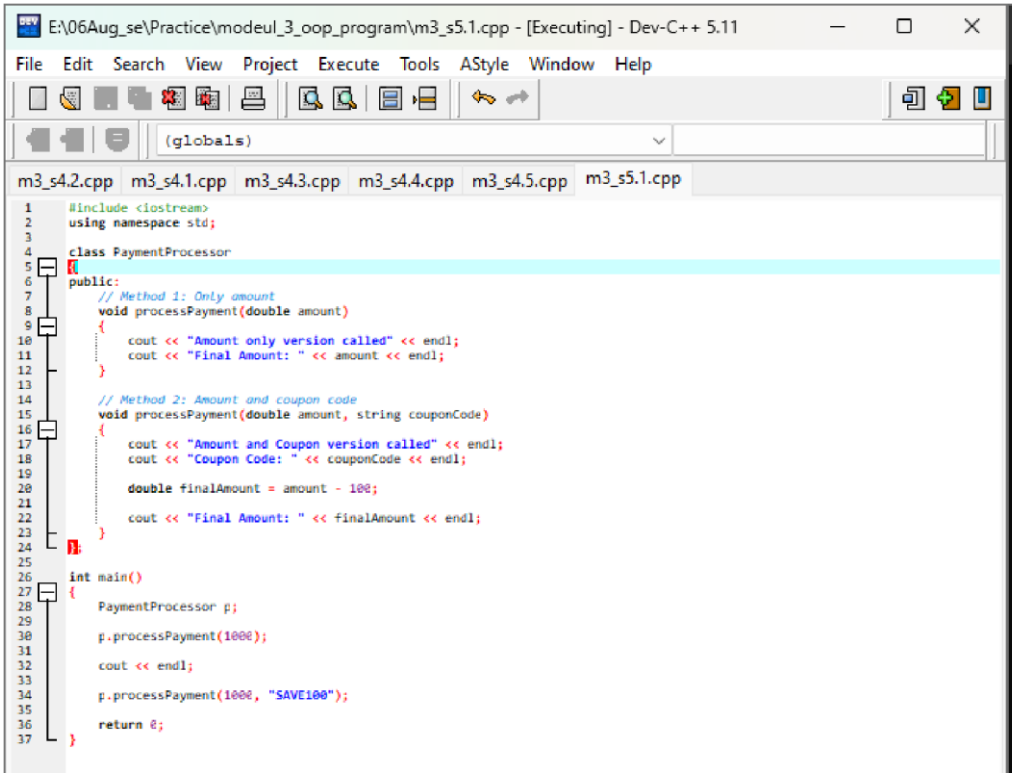
```
    return 0;
}
```

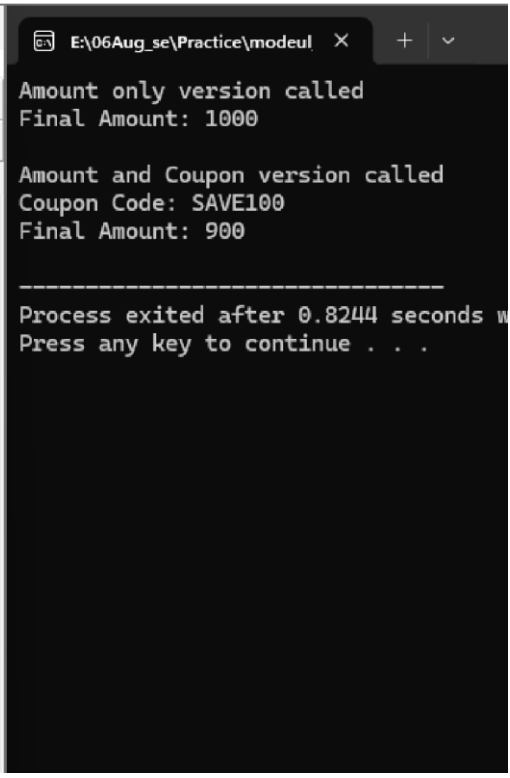
SocialMediaUser

/ | \

YouTuber Podcaster InstagramInfluencer Session-5

Q 1. Create a Java class called PaymentProcessor with two overloaded methods processPayment(): one that takes only an amount, and one that takes amount and a coupon code. Print which version is called and the final amount in each case.





Ans.

```
#include <iostream> using
namespace std;
```

```
class PaymentProcessor
{
public:
    // Method 1: Only amount     void
processPayment(double amount)
    {
        cout << "Amount only version called" << endl;
cout << "Final Amount: " << amount << endl;
    }

    // Method 2: Amount and coupon code
void processPayment(double amount, string
couponCode)
    {
        cout << "Amount and Coupon version called" <<
endl;
        cout << "Coupon Code: " << couponCode << endl;

        double finalAmount = amount - 100;

        cout << "Final Amount: " << finalAmount << endl;
    }
};
```

```
int main()
{
    PaymentProcessor p;

    p.processPayment(1000);

    cout << endl;

    p.processPayment(1000, "SAVE100");

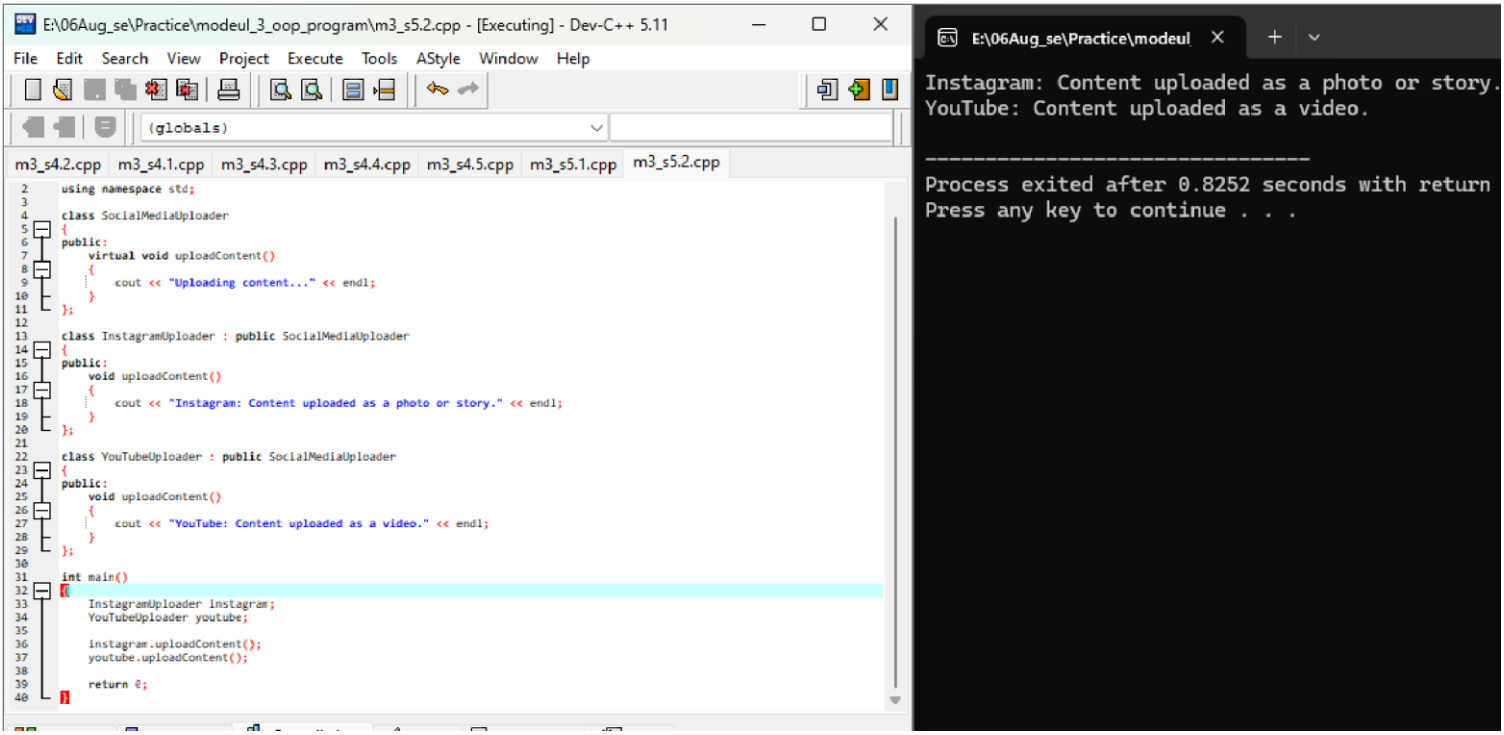
    return 0;
}
```

Q 2. Build two classes, InstagramUploader and YouTubeUploader, each with a method uploadContent(). Both should extend a base class SocialMediaUploader and override uploadContent() to print a message showing how uploading works differently for Instagram and YouTube.

Ans.

```
#include <iostream> using
namespace std;

class SocialMediaUploader
{
public:     virtual void
```




```
uploadContent()
    {
        cout << "Uploading content..." << endl;
    }
};
```

```
class InstagramUploader : public
SocialMediaUploader
{
public:     void
uploadContent()
    {
        cout << "Instagram: Content uploaded as a
photo or story." << endl;
    }
};

class YouTubeUploader : public
SocialMediaUploader
{
public:
    void uploadContent()
    {
        cout << "YouTube: Content uploaded as a
video." << endl;
    }
};

int main()
{
```

```
    InstagramUploader instagram;
    YouTubeUploader youtube;

    instagram.uploadContent();
youtube.uploadContent();

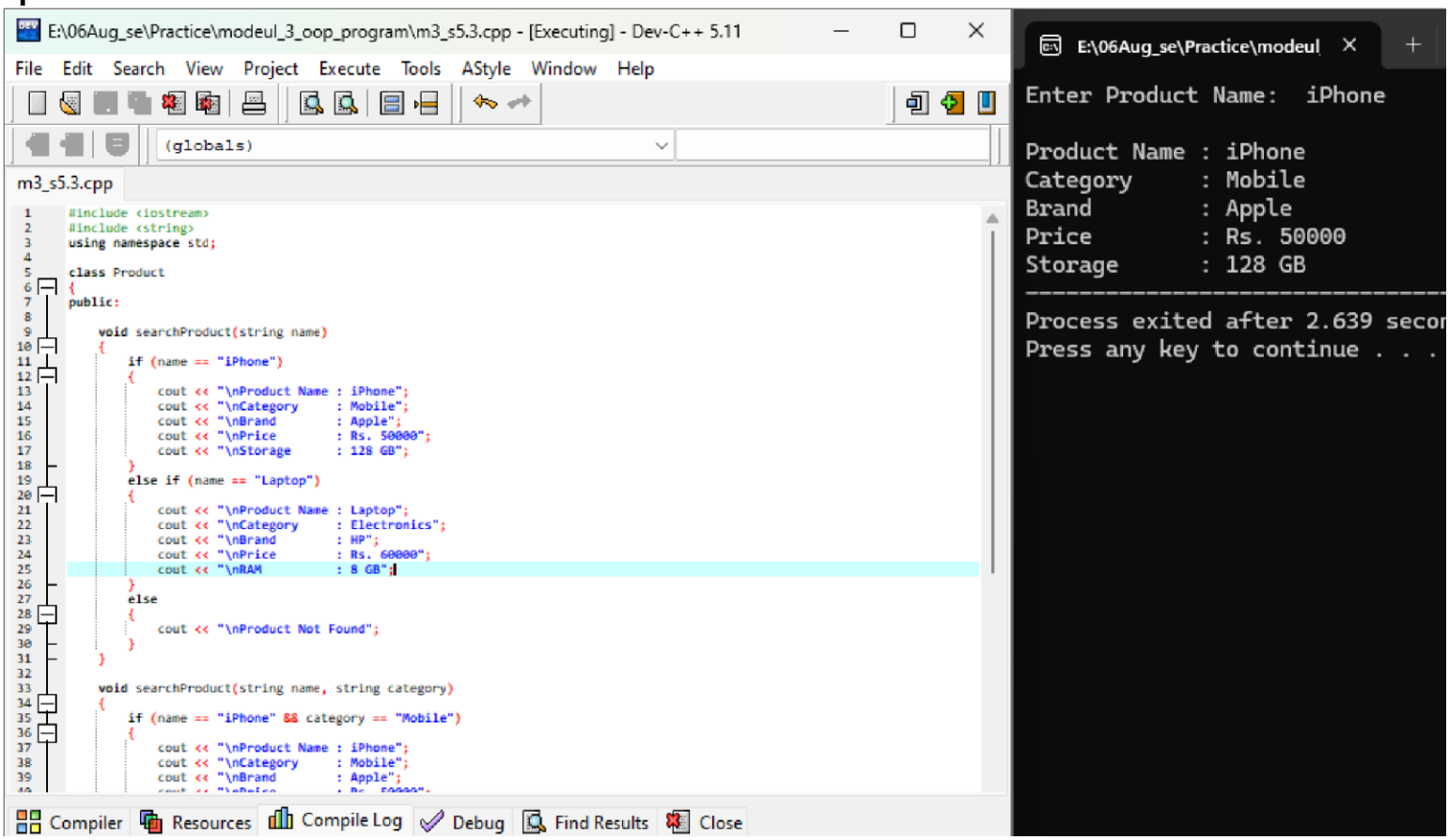
    return 0;
}
```

Q 3 Write a function in Java or Python that simulates a Flipkart-style search: overload a method searchProduct() to allow searching by product name or by product name and category. Demonstrate both usages with sample data.

Ans.

```
#include <iostream>
#include <string>
using namespace std;

class Product
{
public:
```




```
    void searchProduct(string name)
    {
        if (name == "iPhone")
        {
            cout << "\nProduct Name : iPhone";
cout << "\nCategory     : Mobile";
```

```
cout << "\nBrand        : Apple";             cout
<< "\nPrice        : Rs. 50000";             cout
<< "\nStorage      : 128 GB";
        }
        else if (name == "Laptop")
        {
            cout << "\nProduct Name : Laptop";
cout << "\nCategory     : Electronics";
cout << "\nBrand        : HP";             cout <<
"\nPrice        : Rs. 60000";             cout <<
"\nRAM          : 8 GB";
        }
else
        {
            cout << "\nProduct Not Found";
        }
    }
    void searchProduct(string name, string category)
    {
        if (name == "iPhone" && category == "Mobile")
        {
            cout << "\nProduct Name : iPhone";
cout << "\nCategory     : Mobile";
cout << "\nBrand        : Apple";             cout
```

```
<< "\nPrice        : Rs. 50000";            cout
<< "\nStorage      : 128 GB";
        }
else
        {
            cout << "\nProduct Not Found";
        }
    }
};

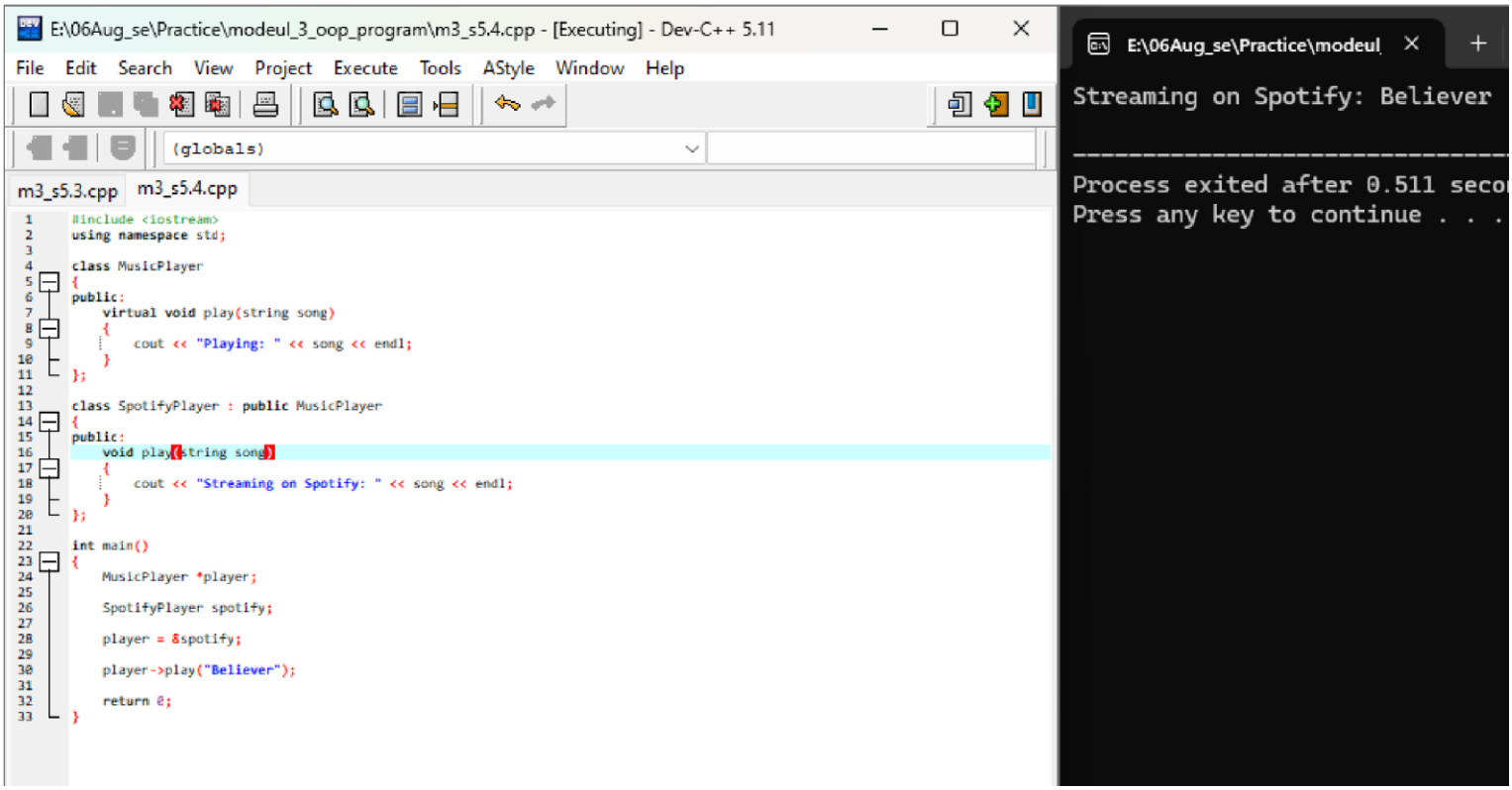
int main()
{
    Product p;
string name;

    cout << "Enter Product Name: ";
cin >> name;
    p.searchProduct(name);

    return 0;
}
```

Q 4. Given this code: class MusicPlayer { void play(String song) { System.out.println ("Playing: " + song); } } class SpotifyPlayer extends MusicPlayer { void play(String song) { System.out.println("Streaming on Spotify: " + song); } } — Create an object of type MusicPlayer but assign it a SpotifyPlayer instance, then call play(). Explain the output.

Hint: This tests runtime polymorphism (overriding) and dynamic method dispatch. Ans.



#include <iostream> using namespace std; class MusicPlayer

```
{
public:
    virtual void play(string song)
    {
        cout << "Playing: " << song << endl;
    }
};

class SpotifyPlayer : public MusicPlayer
{
public:     void
play(string song)
    {
        cout << "Streaming on Spotify: " << song << endl;
    }
};

int main()
{
    MusicPlayer *player;
    SpotifyPlayer spotify;

    player = &spotify;
```

```
    player->play("Believer");

    return 0;
}
```

Session-6

Q 1. Create a class called Song in your preferred OOP language with private properties title and artist. Add public getter and setter methods to access and modify these properties, then create an object and update its title.

Ans.

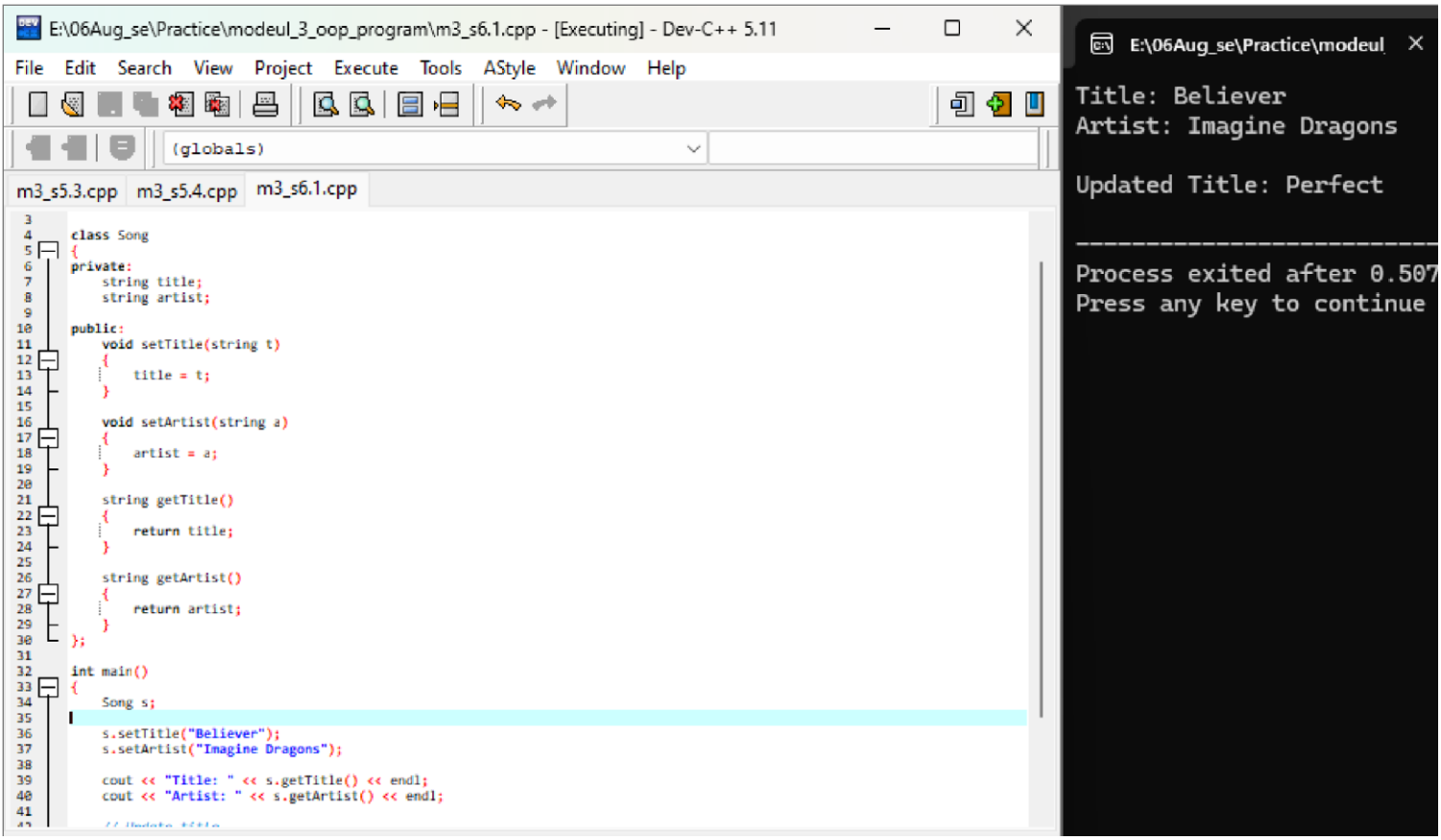
#include <iostream> using namespace std; class Song

{

private:

string title;

string artist;



public: void setTitle(string t)

{ title = t;

```
    }

    void setArtist(string a)
    {         artist
= a;
    }

    string getTitle()
    {         return
title;
    }

    string getArtist()
    {         return
artist;     }
};

int main()
{
    Song s;

    s.setTitle("Believer");
    s.setArtist("Imagine Dragons");

    cout << "Title: " << s.getTitle() << endl;
cout << "Artist: " << s.getArtist() << endl;
```

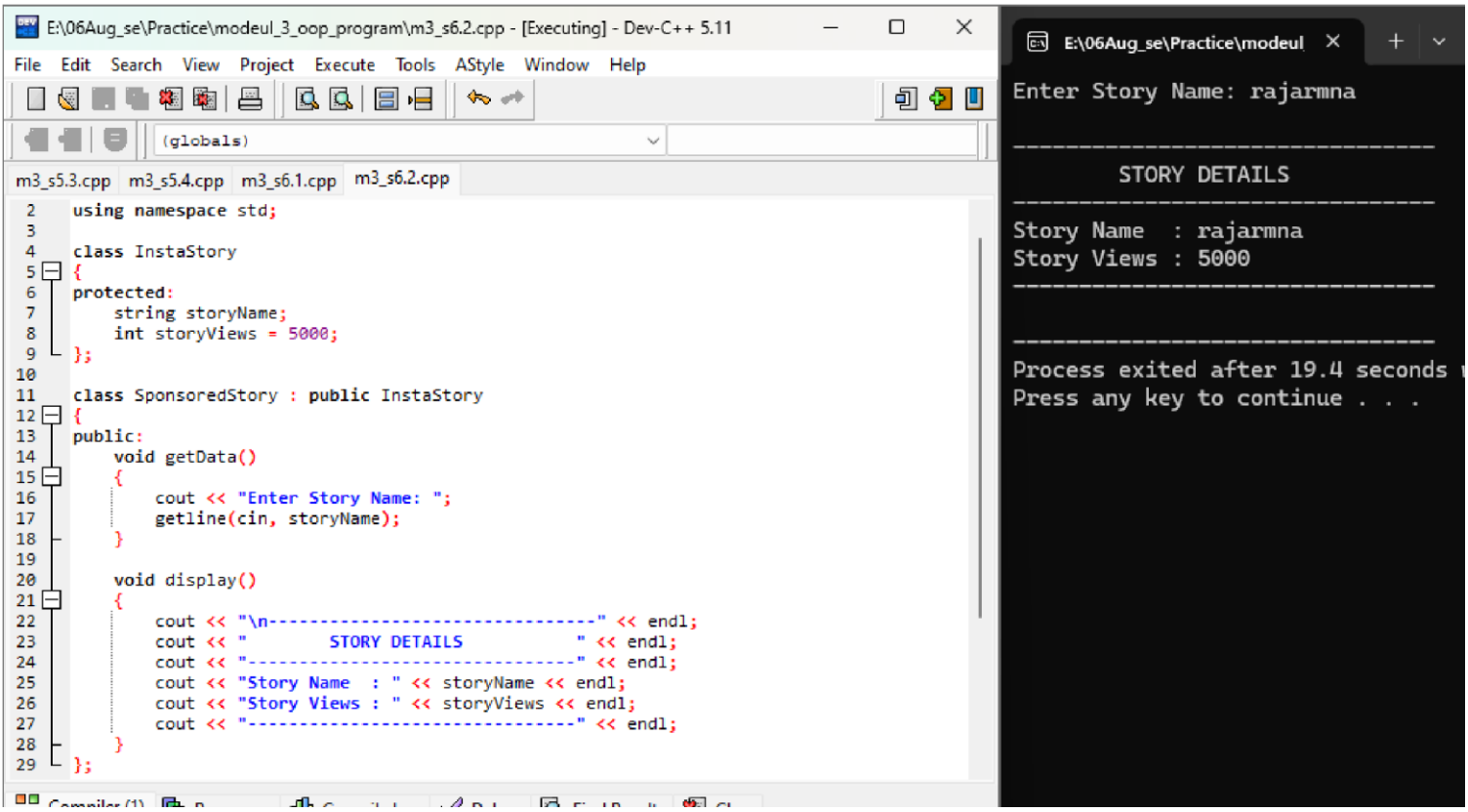
```
    // Update title
    s.setTitle("Perfect");

    cout << "\nUpdated Title: " << s.getTitle() << endl;

    return 0;
}
```

Q 2. Build a class called InstaStory with a protected property storyViews. Create a subclass called SponsoredStory that can access and display the storyViews value.

Ans.




```
#include <iostream> using
namespace std;

class InstaStory
{
protected:
    string storyName;
    int storyViews = 5000;
```

```
};

class SponsoredStory : public InstaStory
{
public:     void
getData()
    {
        cout << "Enter Story Name: ";
getline(cin, storyName);
    }

    void display()
    {        cout << "\n-----------------------------" << endl;
cout << "          STORY DETAILS              " << endl;
cout << "-----------------------------" << endl;         cout
<< "Story Name  : " << storyName << endl;         cout
<< "Story Views : " << storyViews << endl;         cout
<< "-----------------------------" << endl;     }
};

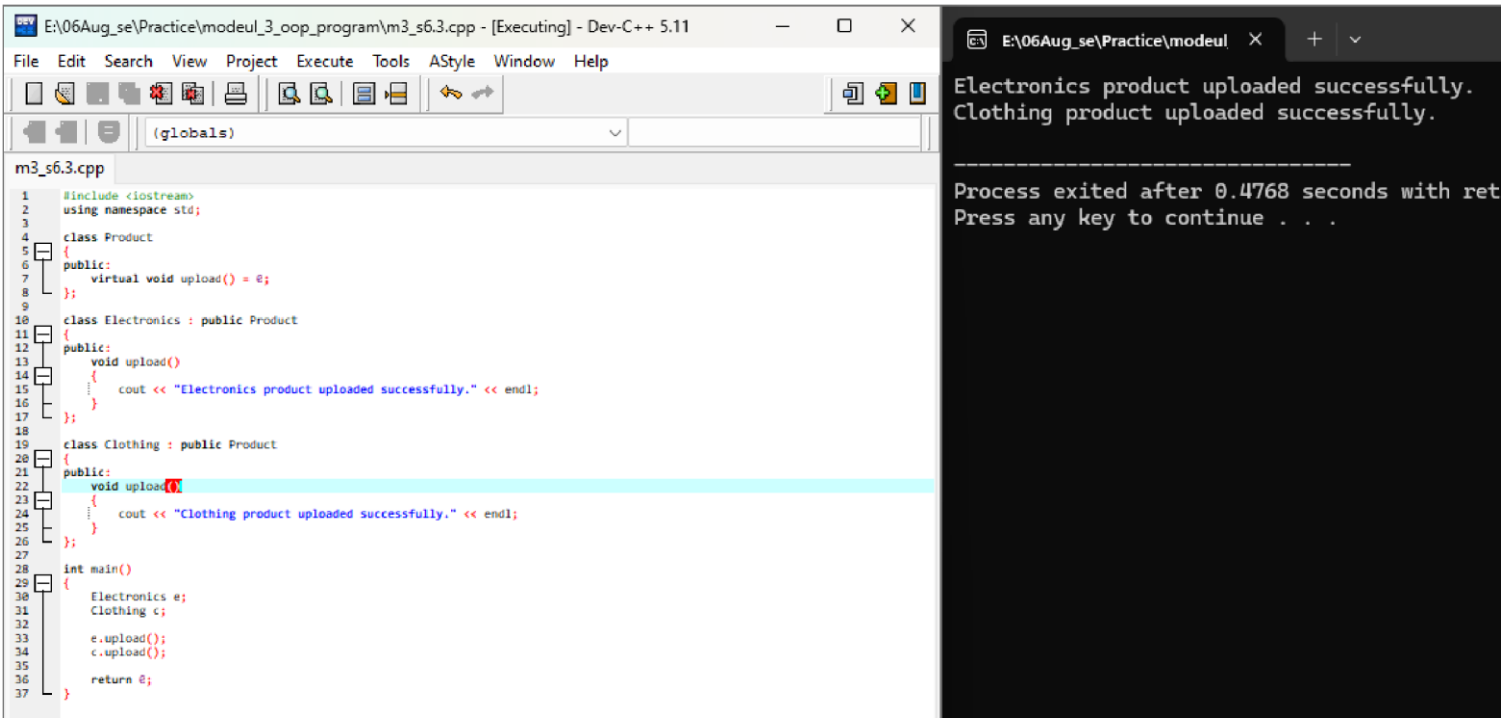
int main()
{
    SponsoredStory s;

    s.getData();
    s.display();
```

```
    return 0;
}
```

Q 3. Imagine a Flipkart-like app: create an abstract class Product with an abstract method upload(). Then, create two subclasses, Electronics and Clothing, that each implement the upload() method to print a different upload message.

Ans.




```
#include <iostream>
```

```
using namespace std;

class Product
{
public:
    virtual void upload() = 0;
};

class Electronics : public Product
{
public:     void
upload()
    {
        cout << "Electronics product uploaded
successfully." << endl;
    }
};

class Clothing : public Product
{
public:     void
upload()
    {
        cout << "Clothing product uploaded
successfully." << endl;
    }
```

```
};

int main()
{
    Electronics e;
    Clothing c;

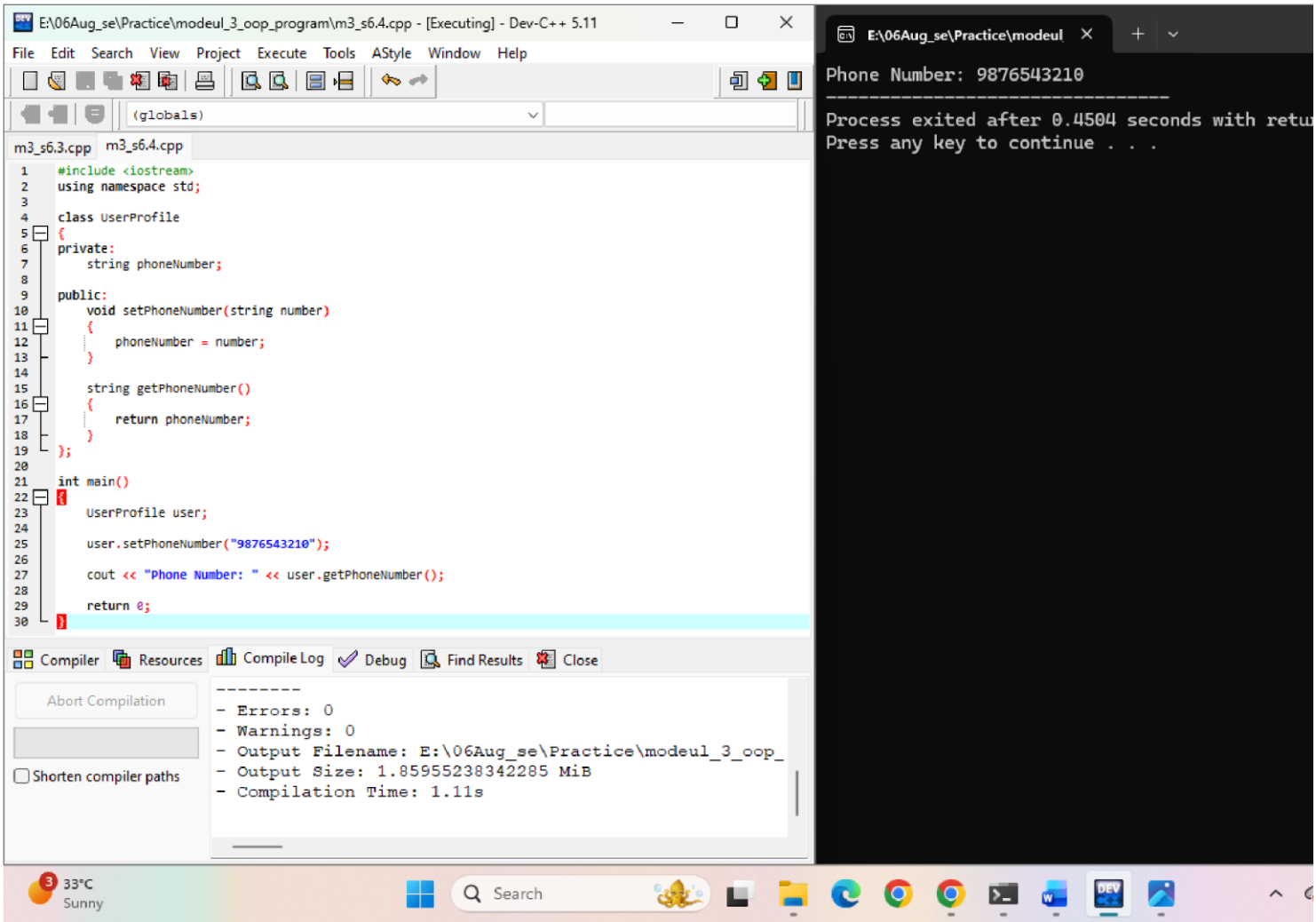
    e.upload();
    c.upload();

    return 0;
}
```

Q 4. Refactor the following code so that the user's phone number in the UserProfile class is private and can only be set or retrieved using public methods.

Hint: Add private access modifier to the phone number and create setPhoneNumber() and getPhoneNumber() methods.

Ans.



#include <iostream> using namespace std; class UserProfile

```
{
private:
    string phoneNumber;

public:
    void setPhoneNumber(string number)
    {
        phoneNumber = number;
    }

    string getPhoneNumber()
    {
        return phoneNumber;
    }
};

int main()
{
    UserProfile user;

    user.setPhoneNumber("9876543210");

    cout << "Phone Number: " <<
user.getPhoneNumber();
    return 0;
}
```

Q 5. Explain in your own words the difference between encapsulation and abstraction, using examples from any social media app you use daily (like Instagram or WhatsApp).

Ans.

Encapsulation and Abstraction are important concepts of Object-Oriented Programming (OOP), but they are used for different purposes.

Encapsulation means wrapping data and methods together in a single unit and protecting the data from direct access.

It controls how the data can be accessed or changed.

Example: In Instagram, our password, private account information, and other personal settings are protected. We cannot directly access or change the internal data.

We use the options provided by Instagram to update our profile or password.

Abstraction means hiding the internal implementation details and showing only the necessary features to the user.

The user does not need to know how the feature works internally.

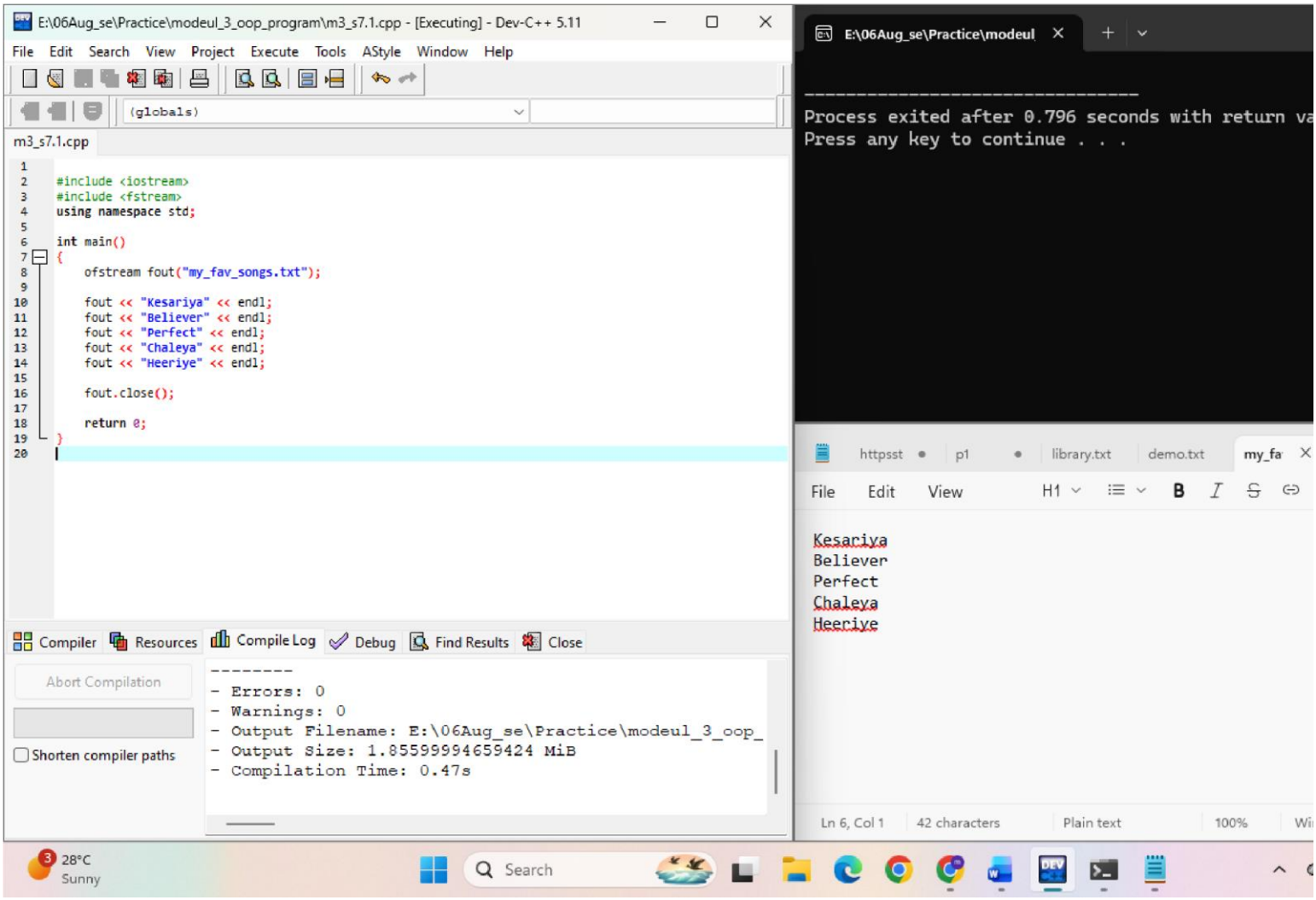
Example: In WhatsApp, when we send a message, we simply type the message and click the Send button.

We do not need to know how WhatsApp processes, encrypts, and delivers the message internally.

Session-7

Q 1. Create a text file called my_fav_songs.txt and write the names of your 5 favorite songs into it using ofstream.

Ans.




```
#include <iostream>
#include <fstream>
using namespace std; int
main()
{
```

```
    ofstream fout("my_fav_songs.txt");

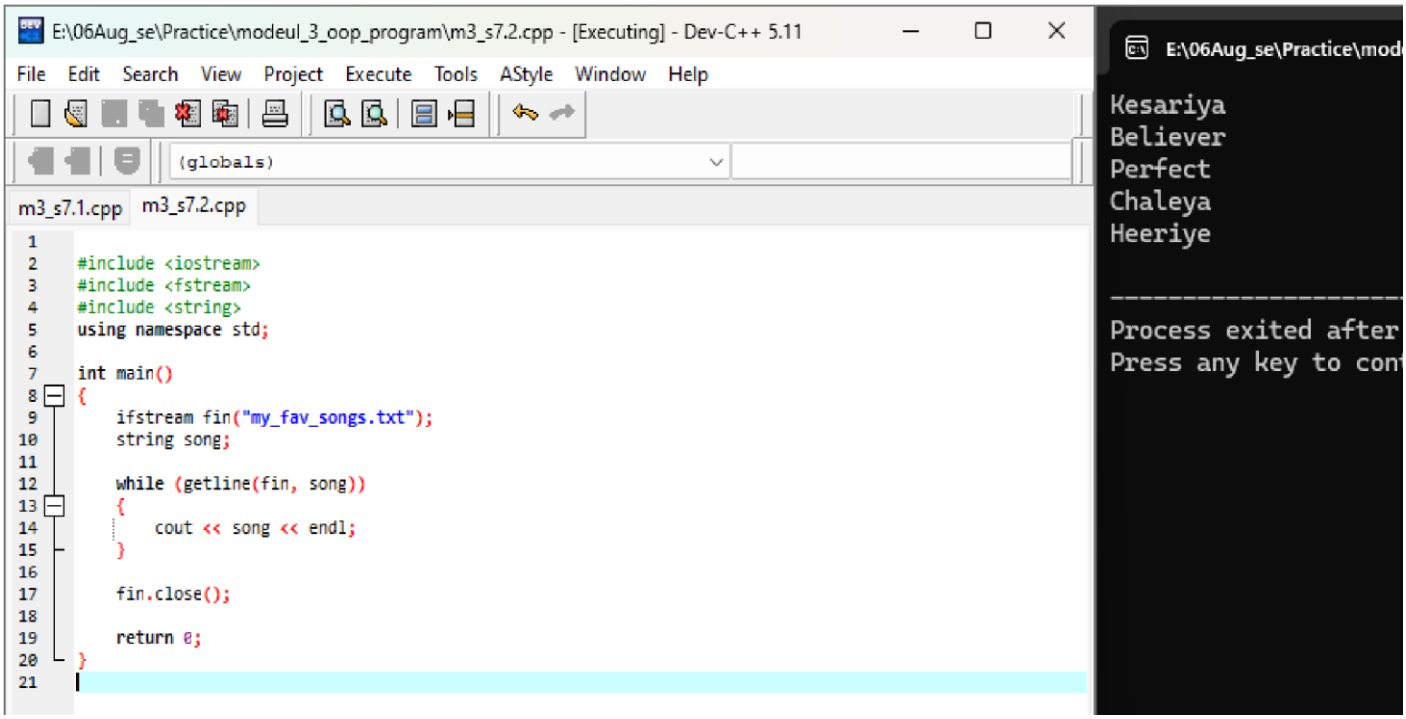
    fout << "Kesariya" << endl;
fout << "Believer" << endl;
fout << "Perfect" << endl;
fout << "Chaleya" << endl;
fout << "Heeriye" << endl;

    fout.close();

    return 0;
}
```

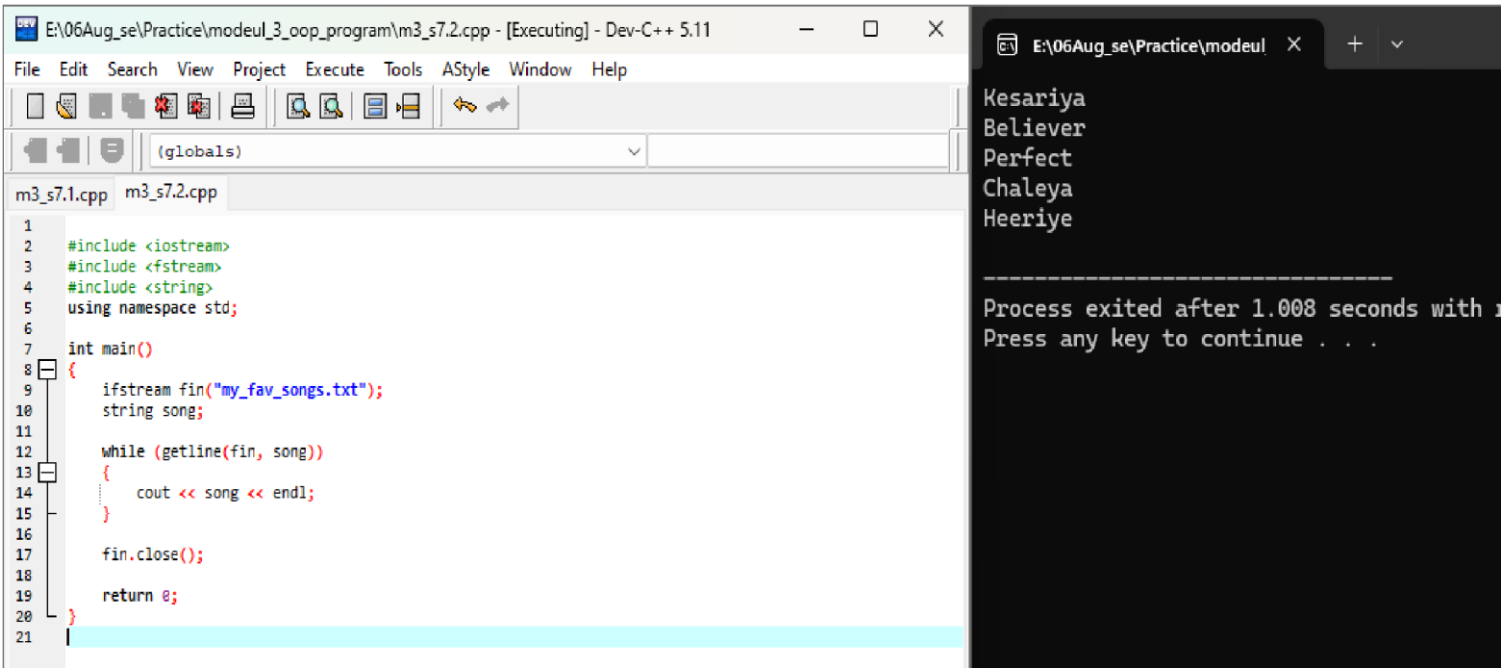
Q 2. Read all song names from my_fav_songs.txt using ifstream and display each song on a new line in the console.

Ans.



Q 3. Modify your code to append a new song name entered by the user to my_fav_songs.txt without overwriting the existing list.

Hint: Open the file in append mode using ofstream. Ans.




```
#include <iostream>
#include <fstream>
#include <string> using
namespace std;

int main()
{
    ifstream fin("my_fav_songs.txt");
    string song;
```

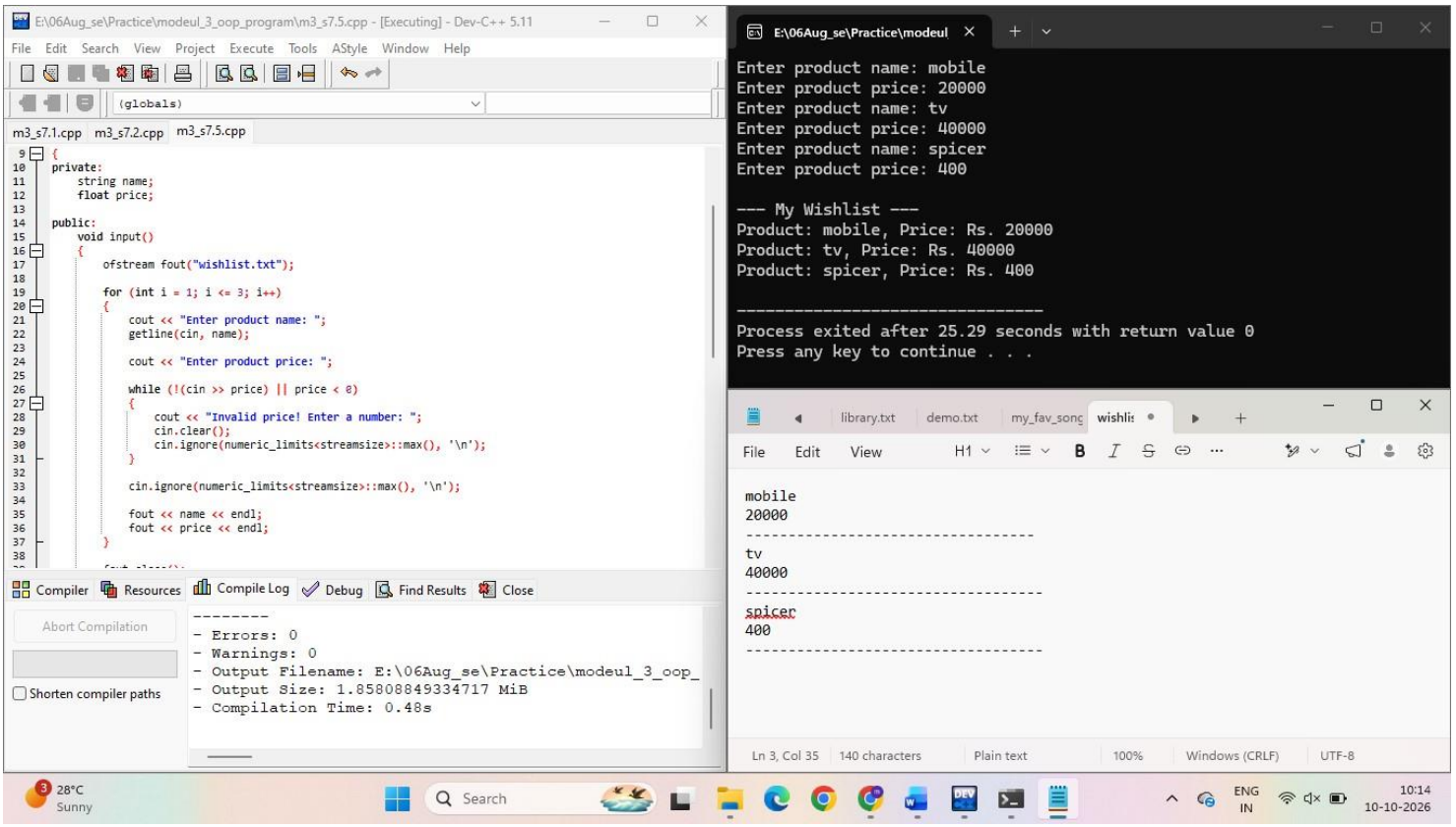
```
    while (getline(fin, song))
    {
        cout << song << endl;
    }

    fin.close();

    return 0;
}
```

Q 4. Build a Flipkart-style wishlist tracker: ask the user to enter 3 product names and prices, save them to a file called wishlist.txt, then read the file and display each product with its price.

Ans.




```
#include <iostream>
#include <fstream>
#include <string>
#include <limits> using
namespace std; class
Wishlist
{
private:
    string name;
float price;

public:
    void input()
```

```
    {
        ofstream fout("wishlist.txt");
         for (int i = 1; i <= 3;
i++)
        {
            cout << "Enter product name: ";
getline(cin, name);

            cout << "Enter product price: ";

            while (!(cin >> price) || price < 0)
            {
                cout << "Invalid price! Enter a number: ";
cin.clear();

cin.ignore(numeric_limits<streamsize>::max(), '\n');
            }
cin.ignore(numeric_limits<streamsize>::max(),
'\n');

            fout << name << endl;
fout << price << endl;
        }
fout.close();
    }

    void display()
```

```
    {         ifstream
fin("wishlist.txt");

        cout << "\n--- My Wishlist ---\n";

        while (getline(fin, name))
        {             if (fin
>> price)
            {

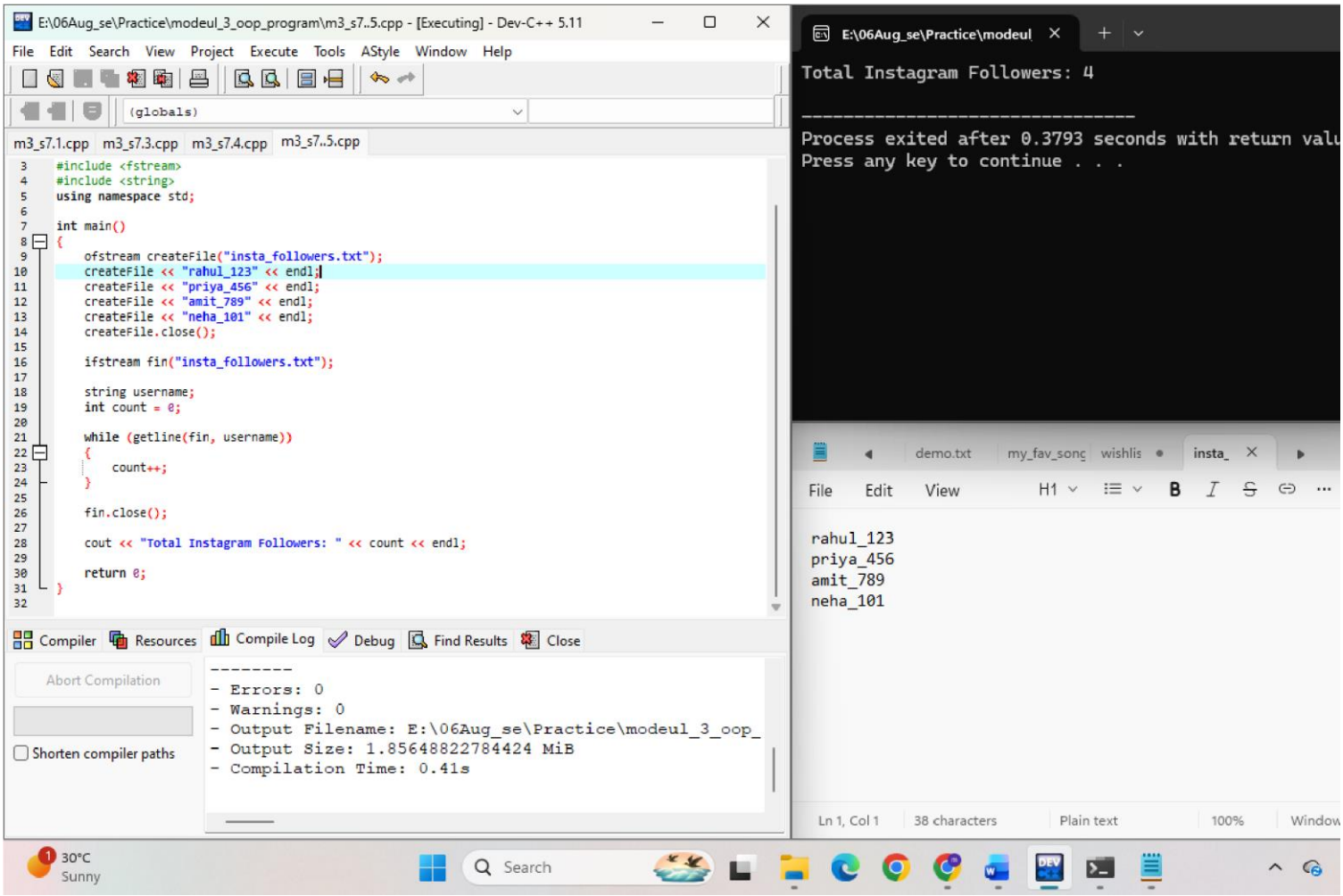
fin.ignore(numeric_limits<streamsize>::max(), '\n');

                cout << "Product: " << name
                    << ", Price: Rs. " << price << endl;
            }
else            {
break;
            }
        }
fin.close();
    }
};

int main()
{
    Wishlist w;
```

```
    w.input();
    w.display();

    return 0;
}
```



Q 5. Given a file called insta_followers.txt containing Instagram usernames (one per line), write a program to count and display the total number of followers listed in the file.

Constraint: Do not use any array or vector to store the names—just count as you read. Ans.

```
#include <iostream>
#include <fstream>
#include <string> using
namespace std; int
main()
{
    ofstream createFile("insta_followers.txt");
createFile << "rahul_123" << endl;
createFile << "priya_456" << endl;
createFile << "amit_789" << endl;     createFile
<< "neha_101" << endl;     createFile.close();

    ifstream fin("insta_followers.txt");
string username;     int count = 0;
while (getline(fin, username))
    {
        count++;
    }

    fin.close();

    cout << "Total Instagram Followers: " << count <<
endl;
```

```
    return 0;
}
```